

École Marocaine des Sciences de l'Ingénieur (EMSI)



Rapport de projet de fin d'études

Conception et réalisation d'une solution web et mobile de gestion
et de souscription d'actions

Architecture microservices, portail web interne, souscription mobile d'OPV
et traçabilité

Réalisé par

Sohaib Mokhliss

Encadrement

Souhaib Allout

Structure d'accueil

Eurafric Information (EAI)

Année académique 2025–2026

Dédicaces

Ce travail est dédié à ma famille, pour son soutien constant, sa patience et sa présence tout au long de mon parcours.

Il est également dédié à toutes les personnes qui m'ont encouragé pendant mes études et qui ont contribué, directement ou indirectement, à l'aboutissement de ce projet de fin d'études.

Remerciements

Je tiens à remercier l'ensemble des personnes ayant contribué à la réalisation de ce projet, en particulier les équipes d'accueil d'Eurafric Information pour le cadre professionnel offert, l'encadrant pédagogique, l'encadrant métier, les membres du jury ainsi que l'école pour leurs orientations, leurs retours et leur accompagnement tout au long du stage.

Je remercie également les collaborateurs qui ont permis de mieux comprendre les enjeux fonctionnels et techniques du sujet, ainsi que toutes les personnes ayant contribué, directement ou indirectement, à la progression de ce travail.

Résumé

Ce rapport présente la conception et la réalisation d'une solution bancaire composée d'une plateforme web interne et d'une application mobile de souscription OPV. Le projet repose sur une architecture microservices Spring Boot, une interface web React, un client mobile React Native, une base PostgreSQL, un routage centralisé par API Gateway, une découverte de services via Eureka, ainsi qu'une traçabilité fondée sur Kafka et un service d'audit. L'authentification mobile des clients a été intégrée à Keycloak afin de se rapprocher du modèle d'identité visé par l'environnement cible.

La solution réalisée permet d'authentifier les utilisateurs internes, de séparer les responsabilités entre administrateurs, agents et spectateurs, de gérer le cycle de vie complet d'une demande, d'administrer le catalogue OPV, de consulter les journaux d'activité et d'exporter les dossiers finalisés au format Excel. Côté mobile, elle permet au client de se connecter, de consulter les OPV ouvertes, à venir et clôturées, de soumettre une souscription avec pièces justificatives et de suivre l'état de sa demande. Une attention particulière a été portée à la cohérence entre services, à la sécurité des accès, à la fiabilité des traitements et à la qualité de l'expérience web et mobile.

Le rapport se concentre sur la solution effectivement réalisée et démontrée dans le cadre du PFE : l'application web interne, l'application mobile client, leur backend mutualisé, leur architecture, leurs fonctionnalités métier et les principales améliorations apportées au cours du projet.

Mots clés : microservices, Spring Boot, React, React Native, PostgreSQL, Kafka, Keycloak, audit, OPV, workflow bancaire

Abstract

This report presents the design and implementation of a banking solution composed of an internal web platform and a complementary mobile application dedicated to the management of subscription requests in a banking context. The project was carried out with a Spring Boot microservices backend, a React web interface, a React Native mobile client, PostgreSQL persistence, an API gateway, Eureka service discovery, Kafka-based event propagation, and a dedicated audit service. Mobile client authentication was integrated with Keycloak in order to reflect the target identity model more realistically.

The delivered solution supports internal authentication and role-based access control for administrators, agents, and spectators, as well as a separate mobile client journey for OPV consultation and subscription. It covers request lifecycle management, administrative supervision, OPV catalog management, mobile subscription submission with supporting documents, export of finalized requests to Excel, and functional audit logging. The work also included several hardening actions on inter-service consistency, internal endpoint protection, request reassignment validation, startup automation, and documentation of the most important technical and functional decisions.

The report focuses on the delivered web and mobile scope that was actually implemented and demonstrated : the internal platform, the mobile OPV subscription application, their shared backend architecture, and the main implementation and validation results obtained during the project.

Keywords: microservices, Spring Boot, React, React Native, PostgreSQL, Kafka, Keycloak, audit, IPO subscription workflow

Table des matières

Dédicaces	i
Remerciements	ii
Résumé	iii
Abstract	iv
Table des matières	v
Liste des figures	viii
Liste des tableaux	x
Liste des abréviations	xi
Introduction générale	1
1 Présentation du cadre du projet	3
1.1 Organisation d'accueil	3
1.2 Présentation de BANK OF AFRICA – BMCE Group	3
1.2.1 Conseil d'administration	3
1.2.2 Gouvernement d'entreprise	4
1.2.3 Actionnariat	5
1.3 Existant avant le projet	6
1.4 Besoin de digitalisation du processus	6
1.5 Solution proposée	7
2 Spécification des besoins	8
2.1 Recueil et analyse du besoin	8
2.2 Cahier des charges	8

2.2.1	Objectif général	8
2.2.2	Solution à réaliser	8
2.2.3	Applications à développer	9
2.2.4	Architecture backend à mettre en place	9
2.2.5	Acteurs et rôles pris en charge	9
2.2.6	Exigences fonctionnelles du portail web	10
2.2.7	Exigences fonctionnelles de l'application mobile	10
2.2.8	Exigences techniques	10
2.2.9	Exigences non fonctionnelles	10
2.2.10	Exigences DevOps	11
2.2.11	Livrables attendus	11
2.2.12	Critères de réussite	11
2.3	Contraintes et faisabilité	12
2.4	Démarche méthodologique adoptée	12
2.5	Planification du projet	13
3	Conception du système	14
3.1	Architecture générale de la solution	14
3.2	Architecture fonctionnelle	15
3.3	Organisation technique du projet	16
3.4	Outils et technologies	16
3.5	Justification des choix techniques	19
3.6	Exemple d'outillage technique	19
3.7	Analyse fonctionnelle	19
3.7.1	Acteurs et périmètre fonctionnel	19
3.7.2	Diagrammes de cas d'utilisation	21
3.8	Conception	23
3.8.1	Conception statique	24
3.8.2	Conception dynamique	26
3.8.3	Conception de la navigation	30
4	Réalisation du système	33
4.1	Mise en oeuvre fonctionnelle	33
4.2	Fonctionnalités principales	33
4.2.1	Authentification et rôles	33

4.2.2	Gestion du workflow métier	33
4.2.3	Consultation et filtrage	34
4.2.4	Export des résultats	34
4.2.5	Audit et supervision	34
4.2.6	Durcissement sécurité et fiabilité	34
4.2.7	Démarche de réalisation	34
4.2.8	Scénario de référence retenu	35
4.2.9	Répartition de l'implémentation	35
4.3	Méthode de validation	36
4.4	Scénario fonctionnel de référence	36
4.4.1	Étape 1 : création de la souscription côté mobile	36
4.4.2	Étape 2 : prise en charge et traitement côté agent	36
4.4.3	Étape 3 : supervision et consultation finale	38
4.5	Synthèse des résultats	38
	Conclusion générale	40
	Bibliographie et Nétographie	41
	ANNEXES	41
	A Industrialisation et environnement DevOps	42
	B Exemples Techniques	48

Table des figures

1.1	Conseil d'administration de BANK OF AFRICA – BMCE Group	4
1.2	Gouvernement d'entreprise de BANK OF AFRICA – BMCE Group	5
1.3	Structure de l'actionnariat de BANK OF AFRICA – BMCE Group	6
3.1	Architecture technique globale de la solution web et mobile	15
3.2	Principales technologies mobilisées dans le projet	18
3.3	Diagramme des acteurs de la solution	20
3.4	Cas d'utilisation associés aux fonctions d'administration	21
3.5	Cas d'utilisation associés au traitement des demandes	22
3.6	Cas d'utilisation de l'application mobile client	23
3.7	Vue statique de la gestion des identités et des profils	24
3.8	Vue statique du domaine des demandes et des OPV	25
3.9	Vue statique des services transverses d'audit et d'export	26
3.10	Séquence d'authentification sur le portail web interne	27
3.11	Séquence d'authentification mobile via Keycloak	28
3.12	Séquence de souscription d'une OPV depuis l'application mobile	29
3.13	Séquence de prise en charge et de traitement d'une demande	30
3.14	Navigation du portail web côté administration	31
3.15	Navigation du portail web pour les parcours agent, spectateur et consultation métier	31
3.16	Shell de navigation de l'application mobile	31
3.17	Parcours mobile dédié à la souscription OPV	32
4.1	Souscription OPV côté mobile : authentification, choix de l'offre, saisie et finalisation	37
4.2	Traitement du dossier côté agent : file d'attente, pièces justificatives et détails métier	37
4.3	Fin de chaîne : supervision administrative, audit et consultation/export des dossiers finalisés	38

A.1	Principales briques de l’usine DevOps du projet	43
A.2	Vue d’ensemble des jobs Jenkins validés pour les services de l’environnement dev	45
A.3	Tableau de bord Grafana utilisé pour suivre l’état général de la plateforme	46

Liste des tableaux

2.1	Planification synthétique du projet	13
3.1	Technologies principales du projet	17
4.1	Répartition des responsabilités dans l'implémentation	35
4.2	Synthèse des validations réalisées	39
A.1	Composants de l'usine DevOps	43
A.2	VMs principales du laboratoire DevOps	44

Liste des abréviations

Acronyme	Signification
API	Application Programming Interface
BMCE	Banque Marocaine du Commerce Extérieur
EAI	Eurafric Information
CI/CD	Continuous Integration / Continuous Delivery
DNS	Domain Name System
GPU	Graphics Processing Unit
GRUB	Grand Unified Bootloader
IP	Internet Protocol
LXC	Linux Containers
SSH	Secure Shell
VE	Virtual Environment
VM	Virtual Machine
HTTP	HyperText Transfer Protocol
IAM	Identity and Access Management
JWT	JSON Web Token
OPV	Offre Publique de Vente
PFE	Projet de fin d'études
REST	Representational State Transfer
SI	Système d'information
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience

Introduction générale

Présentation du sujet

Le projet porte sur la conception et la réalisation d'une solution bancaire articulée autour de deux canaux complémentaires : une plateforme web interne de traitement métier et une application mobile client dédiée à la souscription OPV. L'objectif principal est de remplacer une logique de manipulation de données de test ou de traitement dispersé par une application cohérente, centralisée et structurée autour d'un cycle de vie métier explicite.

La solution mise en oeuvre permet de créer, traiter, superviser, consulter et exporter les demandes selon un modèle de rôles clairement séparé côté web, tout en offrant côté mobile un parcours client centré sur la consultation des OPV et la soumission d'une souscription. Elle répond ainsi à un besoin de digitalisation, de traçabilité et de maîtrise opérationnelle au sein d'un environnement informatique proche des contraintes d'un SI bancaire réel.

Problématique

Dans un contexte de traitement sensible de dossiers financiers, plusieurs difficultés apparaissent rapidement lorsqu'aucun cadre applicatif unifié n'est mis en place :

- Absence de workflow clair entre création, prise en charge et décision ;
- Faible traçabilité des actions réalisées par les utilisateurs ;
- Risque d'incohérence entre les services ou entre les rôles ;
- Difficulté à sécuriser les accès et à restreindre correctement la visibilité des données ;
- Manque d'outils adaptés pour la supervision et l'export des dossiers finalisés ;
- Absence d'un parcours mobile cohérent pour les clients souhaitant souscrire à une OPV.

La problématique centrale du projet peut donc être formulée ainsi : *comment concevoir une solution fiable et sécurisée combinant un portail web interne et une application mobile client, afin de gérer le cycle de vie complet des demandes et souscriptions d'actions, tout en tenant compte des contraintes d'un système distribué moderne ?*

Objectifs

Les objectifs fonctionnels et techniques retenus pour ce projet sont les suivants :

- Mettre en place un cycle de vie explicite des demandes : `PENDING` → `IN_PROGRESS` → `APPROVED` ou `REJECTED` ;
- Fournir une interface web adaptée aux rôles `ADMIN`, `AGENT` et `SPECTATOR` ;
- Fournir une application mobile client permettant la consultation des OPV et la soumission d'une souscription ;
- Garantir l'authentification par jeton JWT et l'autorisation par rôle ;
- Intégrer l'authentification mobile client via Keycloak ;
- Assurer la journalisation des actions critiques ;
- Permettre l'export des dossiers finalisés au format Excel ;
- Structurer la solution autour de microservices Spring Boot, d'un front-end React et d'un client mobile React Native ;
- Documenter les choix d'architecture, les corrections de sécurité et les résultats obtenus.

Sur le plan pédagogique, le projet vise également à consolider la maîtrise de l'architecture distribuée, de la sécurisation inter-services, de la conception d'interfaces de gestion et de la documentation technique d'un produit logiciel complet.

Chapitre 1

Présentation du cadre du projet

1.1 Organisation d'accueil

Eurafric Information (EAI) constitue la structure d'accueil dans laquelle s'inscrit ce projet de fin d'études. Créée comme centre de services informatiques partagés au service du groupe FINANCES.COM et de plusieurs partenaires, l'entreprise intervient dans des domaines variés tels que la banque, l'assurance, la finance et les technologies de l'information [1] [2]. Elle se positionne comme un pôle d'expertise technologique chargé d'accompagner ses partenaires dans la conception, l'exploitation et la sécurisation de leurs systèmes d'information.

Le positionnement d'EAI repose sur une logique de mutualisation des ressources humaines, des outils et des infrastructures afin d'optimiser les coûts et d'accélérer la mise à disposition des solutions numériques. Le cadre du stage s'inscrit donc dans un environnement exigeant, marqué par des besoins de fiabilité, de sécurité, de traçabilité et de continuité de service, particulièrement importants dans le contexte bancaire.

1.2 Présentation de BANK OF AFRICA – BMCE Group

Le projet s'inscrit dans l'écosystème BANK OF AFRICA – BMCE Group, établissement bancaire marocain organisé autour d'une gouvernance structurée. Cette organisation est importante pour comprendre le contexte du sujet : la gestion des demandes d'achat d'actions et des souscriptions OPV concerne un environnement bancaire où les responsabilités, les validations et la traçabilité doivent être clairement maîtrisées.

1.2.1 Conseil d'administration

Le Conseil d'Administration de BANK OF AFRICA – BMCE Group est formé de quinze administrateurs. Il a pour mission essentielle de maintenir l'équilibre entre les intérêts des actionnaires, les perspectives de développement et la protection des déposants. Cette structure de gouvernance encadre les grandes orientations du groupe et contribue à la maîtrise des risques inhérents aux activités bancaires.

Cette figure présente la composition du conseil d'administration de BANK OF AFRICA – BMCE Group, utilisée pour situer le projet dans son contexte institutionnel.

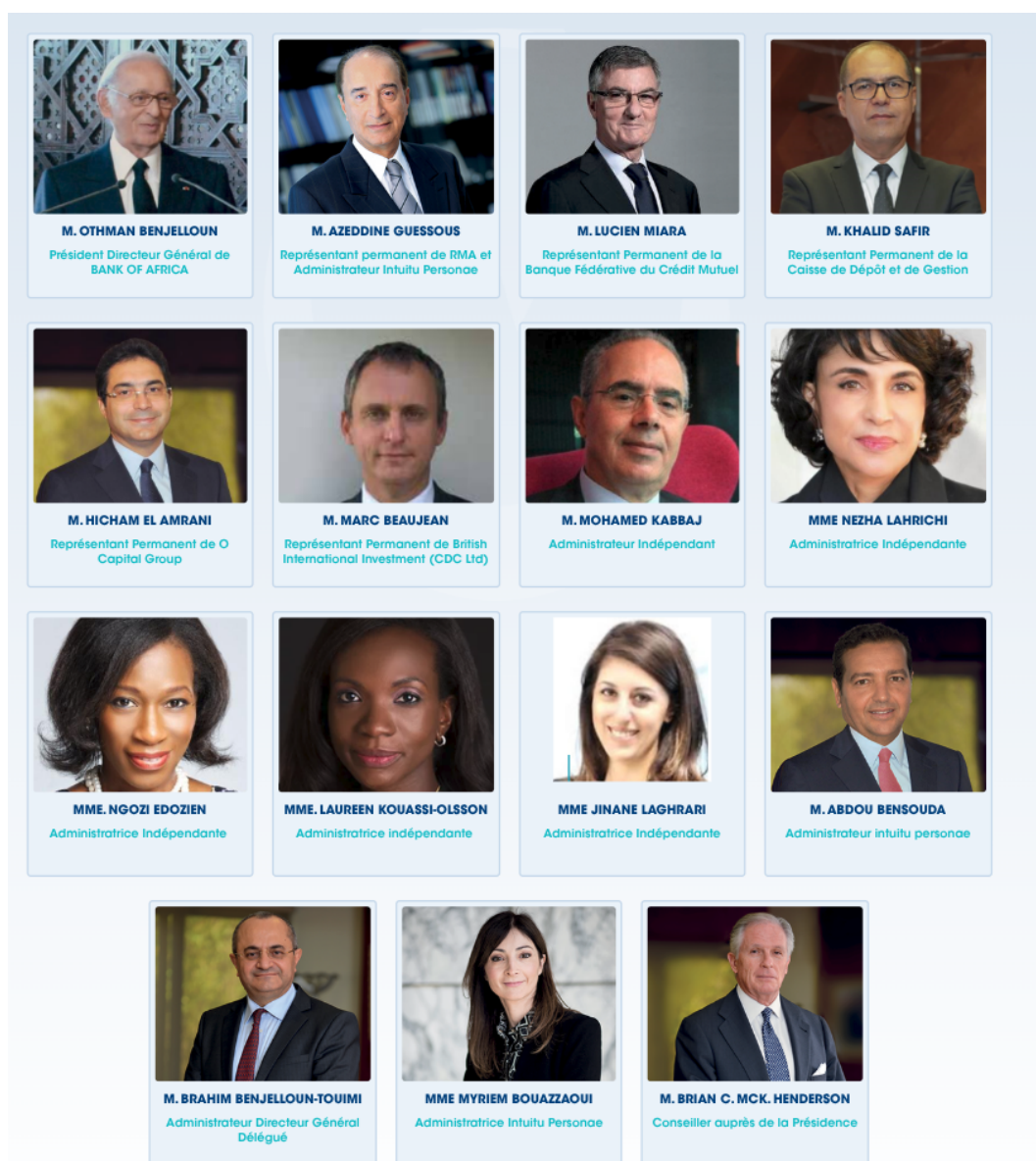


FIGURE 1.1 – Conseil d'administration de BANK OF AFRICA – BMCE Group

1.2.2 Gouvernement d'entreprise

La gouvernance exécutive complète le rôle du Conseil d'Administration en assurant le pilotage opérationnel du groupe. Elle regroupe les responsables chargés de la direction générale, des fonctions transverses, des opérations, des activités au Maroc, de la banque d'investissement et du développement africain. Cette organisation permet de relier les orientations stratégiques aux activités quotidiennes de la banque.

Cette figure illustre l'organisation de la gouvernance exécutive du groupe et les principales responsabilités de pilotage associées.

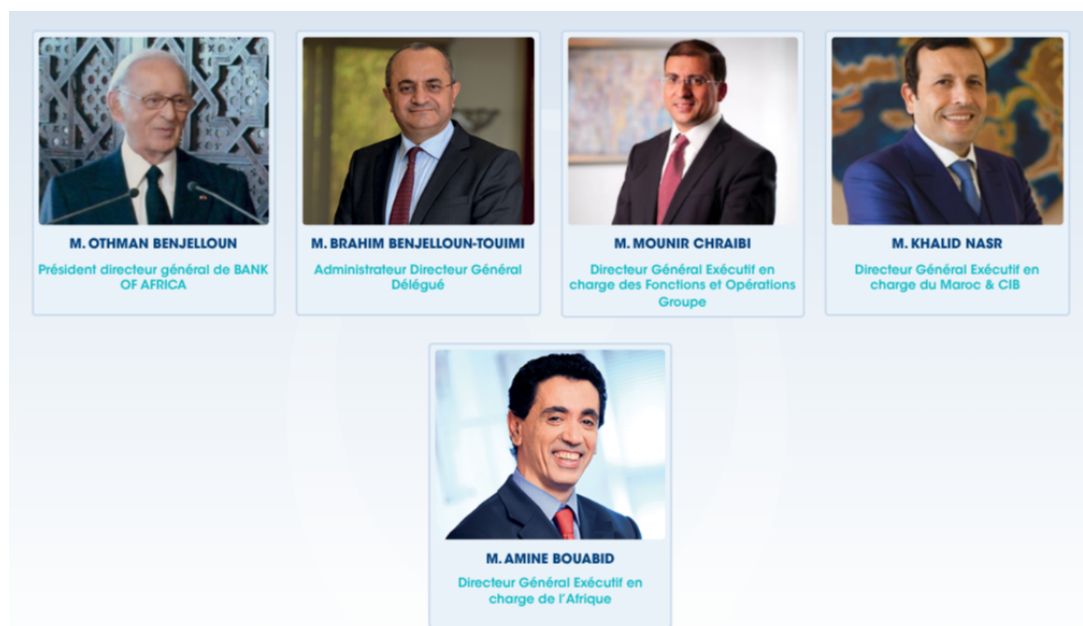


FIGURE 1.2 – Gouvernement d'entreprise de BANK OF AFRICA – BMCE Group

1.2.3 Actionnariat

L'actionnariat du groupe est diversifié et comprend notamment RMA, la Banque Fédérative du Crédit Mutuel, le groupe CDG, O Capital Group, le personnel de BANK OF AFRICA, British International Investment, ainsi que d'autres actionnaires institutionnels et individuels. Cette structure actionnariale illustre le caractère institutionnel du groupe et l'importance d'un processus bancaire encadré pour les opérations liées aux titres.

Cette figure montre la répartition de l'actionnariat du groupe, élément important pour comprendre le cadre bancaire et institutionnel du projet.

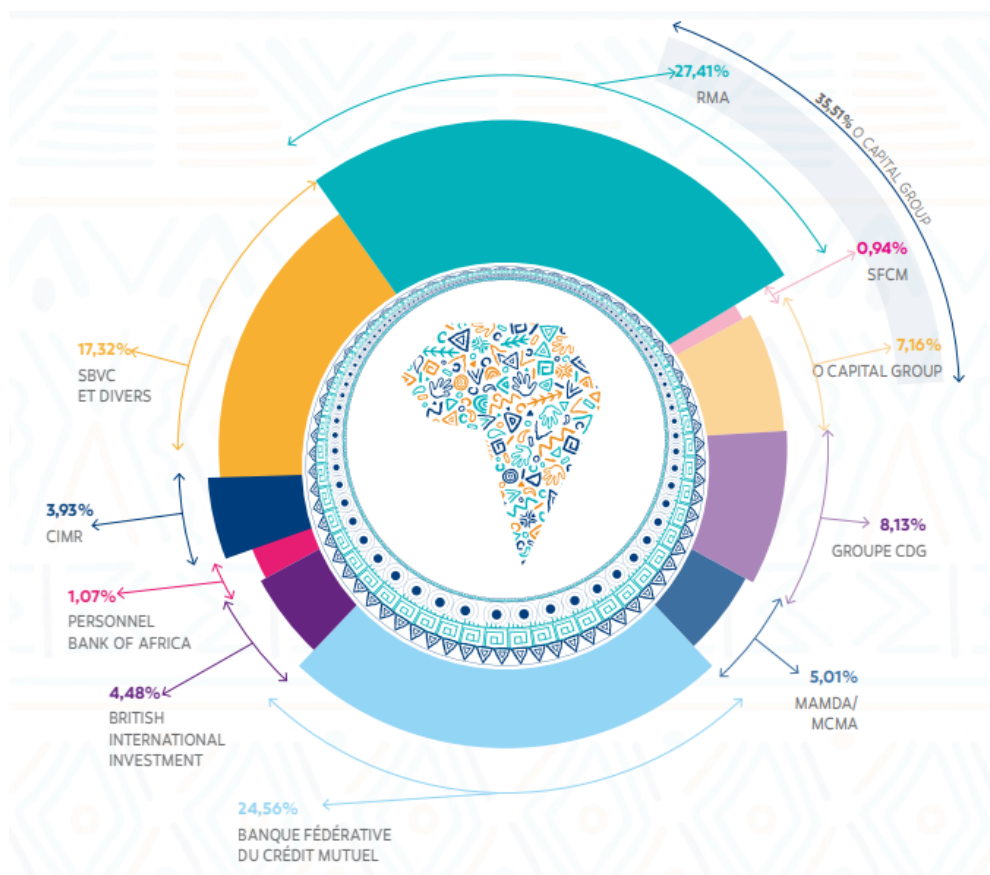


FIGURE 1.3 – Structure de l'actionnariat de BANK OF AFRICA – BMCE Group

1.3 Existant avant le projet

Avant la réalisation de cette solution, le processus lié aux demandes d'achat d'actions et aux souscriptions OPV restait fortement dépendant d'un traitement classique en agence et d'une organisation interne peu centralisée. La commercialisation des offres se faisait au niveau du réseau physique de la banque, tandis que le suivi opérationnel des dossiers reposait sur des traitements séparés, des échanges fragmentés et une visibilité limitée sur l'état global des demandes.

Du point de vue applicatif, l'existant ne proposait pas de chaîne homogène permettant d'assurer à la fois la saisie, le traitement, la supervision, la consultation et la traçabilité du dossier dans un même système. Les rôles opérationnels étaient peu structurés dans l'outil, le parcours client mobile était absent, et l'export ainsi que l'audit restaient difficiles à industrialiser.

1.4 Besoin de digitalisation du processus

Dans ce contexte, le besoin principal est celui de la digitalisation d'un processus métier sensible. Il ne s'agit pas uniquement de dématérialiser un formulaire, mais de construire

une chaîne applicative capable de :

- Centraliser le traitement des demandes d'achat d'actions ;
- Distinguer clairement les rôles métier impliqués dans le cycle de vie du dossier ;
- Offrir un meilleur niveau de traçabilité sur les actions réalisées ;
- Sécuriser l'accès aux données selon le profil utilisateur ;
- Ouvrir un canal mobile cohérent pour la consultation des OPV et la souscription client.

La digitalisation devient donc un levier d'amélioration opérationnelle, de fiabilisation du traitement et de modernisation de l'expérience utilisateur, aussi bien côté interne que côté client.

1.5 Solution proposée

Pour répondre à ce besoin, la solution proposée prend la forme d'une plateforme bancaire articulée autour de deux canaux complémentaires :

- Un portail web interne destiné aux profils **ADMIN**, **AGENT** et **SPECTATOR** ;
- Une application mobile client dédiée à la consultation des OPV et à la souscription.

La solution repose sur une architecture microservices permettant de séparer les responsabilités techniques et métier. Elle couvre l'authentification, la gestion des utilisateurs, la gestion des profils clients, le traitement des demandes, l'export des dossiers finalisés et la centralisation des journaux d'audit.

Le projet vise ainsi à transformer un processus dispersé en un système cohérent, sécurisé et traçable, capable de gérer le cycle de vie complet d'une demande : création, prise en charge, validation ou rejet, consultation et export. Cette proposition constitue le socle de la transformation numérique du processus étudié dans le cadre du stage.

Chapitre 2

Spécification des besoins

2.1 Recueil et analyse du besoin

L'étude préliminaire a eu pour objectif de clarifier le périmètre fonctionnel du projet avant d'engager les choix d'architecture et de réalisation. Cette étape a permis d'identifier les acteurs concernés, les opérations attendues, les contraintes de sécurité et les points de contrôle nécessaires au bon déroulement du processus.

L'analyse du besoin a montré que le système devait couvrir deux dimensions complémentaires :

- Une dimension interne orientée traitement, supervision et exploitation ;
- Une dimension client orientée consultation des OPV et souscription mobile.

Cette double exigence a conduit à retenir une solution unique mais répartie en plusieurs composants spécialisés.

2.2 Cahier des charges

Le cahier des charges du projet formalise de manière concrète la solution à réaliser. Il précise les composants à développer, les technologies à utiliser, les rôles à couvrir ainsi que les exigences techniques et organisationnelles à respecter pour produire une plateforme cohérente, exploitable et évolutive.

2.2.1 Objectif général

L'objectif général est de concevoir et de réaliser une plateforme bancaire digitale permettant de gérer le processus de souscription à des OPV à travers un portail web interne, une application mobile client et une architecture backend en microservices.

2.2.2 Solution à réaliser

La solution attendue dans le cadre du projet comprend les éléments suivants :

- Développer un **portail web interne** destiné aux profils métier de la banque ;
- Développer une **application mobile client** pour la consultation des OPV et la souscription ;
- Mettre en place un **backend en microservices** pour séparer les responsabilités métier ;
- Assurer la **sécurité**, la **traçabilité** et la **cohérence des traitements** ;
- Mettre en place une première base **DevOps** pour l'intégration continue, la qualité de code et la supervision.

2.2.3 Applications à développer

Les applications prévues dans le cahier des charges sont les suivantes :

- Un **portail web d'administration et de traitement** permettant de gérer les utilisateurs, les demandes, les OPV, les journaux d'audit et les exports ;
- Une **application mobile React Native** permettant au client de s'authentifier, de consulter les offres OPV, de visualiser les détails et de soumettre une demande ;
- Une couche **API** unique d'accès aux services à travers une API Gateway ;
- Une couche **services métier** composée de plusieurs microservices spécialisés.

2.2.4 Architecture backend à mettre en place

L'architecture backend retenue dans le cadre du projet doit être basée sur des microservices. Les services attendus sont les suivants :

- **auth-service** pour l'authentification interne, la gestion des comptes et la génération des jetons ;
- **user-service** pour la gestion des utilisateurs métier ;
- **client-service** pour la gestion des profils clients et des données du parcours mobile ;
- **request-service** pour la gestion des OPV, des souscriptions et du cycle de vie des demandes ;
- **audit-service** pour la centralisation des événements d'audit ;
- **export-service** pour la génération et l'export des dossiers finalisés ;
- **api-gateway** comme point d'entrée unique ;
- **eureka-server** pour l'enregistrement et la découverte des services.

2.2.5 Acteurs et rôles pris en charge

La solution doit prendre en charge les acteurs suivants :

- **administrateur** : supervision globale, gestion des utilisateurs, des OPV et des paramètres d'exploitation ;
- **agent** : prise en charge, traitement, validation ou rejet des demandes ;
- **spectateur** : consultation des dossiers finalisés et génération d'exports ;

- **client** : consultation des OPV et souscription via l'application mobile.

2.2.6 Exigences fonctionnelles du portail web

- Authentifier les utilisateurs internes et appliquer les droits selon leur rôle ;
- Afficher un tableau de bord adapté au profil connecté ;
- Permettre la consultation des demandes en attente, en cours, approuvées ou rejetées ;
- Permettre la prise en charge d'une demande par un agent ;
- Permettre l'approbation ou le rejet d'une demande avec mise à jour de son état ;
- Permettre la gestion des utilisateurs métier par l'administrateur ;
- Permettre la gestion des OPV disponibles ;
- Permettre la consultation des journaux d'audit ;
- Permettre l'export des dossiers finalisés.

2.2.7 Exigences fonctionnelles de l'application mobile

- Permettre l'authentification du client via Keycloak ;
- Permettre la consultation de la liste des OPV disponibles ;
- Permettre l'affichage du détail d'une OPV ;
- Permettre la soumission d'une demande de souscription ;
- Permettre la consultation de l'historique ou de l'état des demandes du client ;
- Fournir une interface adaptée à l'usage mobile.

2.2.8 Exigences techniques

- Adopter une architecture microservices permettant de découpler les responsabilités ;
- Développer les services backend avec **Java 17**, **Spring Boot** et **Spring Cloud** ;
- Développer le portail web avec **React**, **TypeScript** et les outils front-end retenus dans le projet ;
- Développer l'application mobile avec **React Native** et **Expo** ;
- Utiliser **PostgreSQL** pour la persistance des données ;
- Utiliser **Kafka** pour les événements d'audit ;
- Utiliser **Keycloak** pour l'authentification client mobile ;
- Centraliser les accès via **API Gateway** et la découverte des services via **Eureka**.

2.2.9 Exigences non fonctionnelles

- Garantir la sécurité des accès par rôle et par jeton ;
- Assurer la traçabilité des opérations critiques ;
- Maintenir une séparation claire des responsabilités entre les services backend ;
- Préserver la cohérence des données entre les différents sous-systèmes ;
- Offrir une base facilement déployable et maintenable dans un environnement de démonstration ;

- Conserver une structure évolutive compatible avec une industrialisation progressive.

2.2.10 Exigences DevOps

- Utiliser **GitLab** pour l'hébergement et la gestion du code source ;
- Utiliser **Jenkins** pour automatiser le build des services ;
- Utiliser **SonarQube** pour l'analyse de qualité et la validation des quality gates ;
- Utiliser **Docker** et **Docker Compose** pour l'exécution locale des composants nécessaires ;
- Mettre en place une base de **supervision** avec les outils de monitoring retenus dans le projet ;
- Préparer la solution à une industrialisation plus avancée via une chaîne CI/CD progressive.

2.2.11 Livrables attendus

Le cahier des charges conduit à la production des livrables suivants :

- Un backend distribué composé des microservices métier ;
- Un portail web interne fonctionnel ;
- Une application mobile React Native fonctionnelle ;
- Une architecture technique documentée ;
- Une base DevOps intégrant build, qualité de code et supervision ;
- Une documentation technique et un rapport de PFE complet.

2.2.12 Critères de réussite

Le projet est considéré comme conforme au cahier des charges si les conditions suivantes sont satisfaites :

- Le portail web permet d'exécuter les parcours métier principaux ;
- L'application mobile permet au client de consulter les OPV et de soumettre une demande ;
- Les microservices communiquent correctement à travers l'architecture mise en place ;
- Les rôles et droits sont correctement appliqués ;
- Les actions sensibles sont journalisées ;
- Les services peuvent être construits dans Jenkins et analysés dans SonarQube ;
- L'ensemble de la solution reste démontrable, maintenable et cohérent avec le besoin métier initial.

2.3 Contraintes et faisabilité

Le projet est soumis à plusieurs contraintes. Sur le plan technique, il impose la coexistence de plusieurs services backend, l'intégration d'un fournisseur d'identité externe pour les clients mobiles, la gestion de documents, la sécurisation des échanges et la mise en place d'une traçabilité exploitable. Sur le plan organisationnel, il s'inscrit dans un calendrier de PFE qui impose de prioriser les fonctionnalités essentielles et de structurer les livrables de manière progressive.

L'étude de faisabilité a néanmoins confirmé que la réalisation était cohérente avec le temps disponible, à condition d'adopter une démarche structurée par phases successives, avec validation incrémentale des composants.

2.4 Démarche méthodologique adoptée

La conduite du projet a été structurée selon un **cycle en Y adapté au contexte du PFE**. Ce choix est cohérent avec la nature du sujet, qui ne se limite pas à une application unique, mais combine plusieurs dimensions devant converger vers une solution intégrée : un portail web interne, une application mobile, un backend en microservices, une couche de sécurité, des mécanismes d'audit ainsi qu'une première base d'industrialisation DevOps.

Dans ce cadre, le cycle en Y permet de faire avancer en parallèle deux axes complémentaires :

- Un axe **fonctionnel**, consacré au cadrage du besoin, à l'identification des acteurs, à la formalisation du cahier des charges et à la définition des parcours métier ;
- Un axe **technique**, consacré aux choix d'architecture, à la structuration en microservices, à la sécurité, aux échanges interservices et aux outils d'exécution et d'industrialisation.

La convergence de ces deux axes s'est traduite par une phase de conception puis de réalisation progressive, dans laquelle les décisions fonctionnelles et techniques ont été rapprochées pour produire une solution cohérente. Cette phase a couvert :

- La définition des services métier, des interactions et des interfaces ;
- Le développement des microservices backend ;
- Le développement du portail web interne ;
- Le développement de l'application mobile ;
- La mise en place des mécanismes de sécurité, d'audit et d'export.

Enfin, la phase de consolidation a porté sur l'intégration et la validation progressive de l'ensemble :

- Intégration progressive des composants ;
- Vérifications fonctionnelles des parcours métier et mobile ;
- Corrections et stabilisation de la plateforme ;
- Mise en place de la base DevOps, du build Jenkins, de l'analyse SonarQube et de la documentation finale.

Le cycle en Y constitue ici un cadre de référence pertinent, car il met l'accent sur la

préparation simultanée du besoin fonctionnel et de l'architecture technique avant leur convergence dans une phase d'intégration. Il correspond donc mieux au projet qu'un cycle en V strict, puisque la difficulté principale ne résidait pas uniquement dans l'enchaînement des phases, mais aussi dans la coordination entre plusieurs sous-systèmes devant fonctionner ensemble de manière cohérente.

2.5 Planification du projet

La planification a été organisée selon une succession de phases logiques, depuis le cadrage initial jusqu'à la finalisation documentaire.

TABLE 2.1 – Planification synthétique du projet

Phase	Contenu principal
Cadrage	Compréhension du contexte métier, identification des acteurs, clarification du besoin
Étude préliminaire	Formalisation du cahier des charges, contraintes, faisabilité et priorisation
Conception	Définition de l'architecture, des rôles, des services et des interactions
Développement backend	Mise en oeuvre des microservices métier, sécurité, audit et export
Développement front-end	Réalisation du portail web interne orienté rôles métier
Développement mobile	Mise en oeuvre du parcours client OPV sur application mobile
Intégration et validation	Vérifications fonctionnelles, tests, corrections et stabilisation
Industrialisation et documentation	Mise en place de la base DevOps et finalisation du rapport

Cette planification a suivi la logique générale du cycle en Y, tout en restant suffisamment souple pour intégrer les ajustements techniques apparus pendant la convergence entre les besoins métier, l'architecture applicative et l'intégration des composants.

Chapitre 3

Conception du système

3.1 Architecture générale de la solution

La solution réalisée suit une architecture microservices centrée sur un point d'entrée unique côté client. Le portail web React destiné aux utilisateurs métier et l'application mobile React Native destinée aux clients communiquent avec une API Gateway qui distribue ensuite les requêtes vers plusieurs services spécialisés. La découverte des services est assurée par Eureka, PostgreSQL est utilisé pour la persistance, Kafka prend en charge la circulation des événements liés à l'audit et Keycloak intervient comme fournisseur d'identité pour l'authentification mobile des clients.

Les composants principaux de la solution sont les suivants :

- **frontend** : interface React destinée aux utilisateurs métier ;
- **application mobile** : client React Native / Expo destiné aux souscriptions OPV ;
- **api-gateway** : point d'entrée unique pour les appels HTTP web et mobile ;
- **auth-service** : authentification, émission de JWT applicatifs et gestion des comptes internes ;
- **user-service** : gestion des utilisateurs opérationnels et de leur état ;
- **client-service** : gestion du profil client et des données de référence du parcours mobile ;
- **request-service** : coeur métier des demandes, des OPV et des souscriptions ;
- **export-service** : génération des exports Excel ;
- **audit-service** : centralisation des logs fonctionnels ;
- **eureka-server** : registre de services ;
- **Keycloak** : authentification des comptes clients mobiles.

Ce schéma présente l'architecture technique globale : le portail web et l'application mobile passent par l'API Gateway, puis les requêtes sont distribuées vers les microservices spécialisés.

Diagramme d'Architecture Technique - Vue Runtime

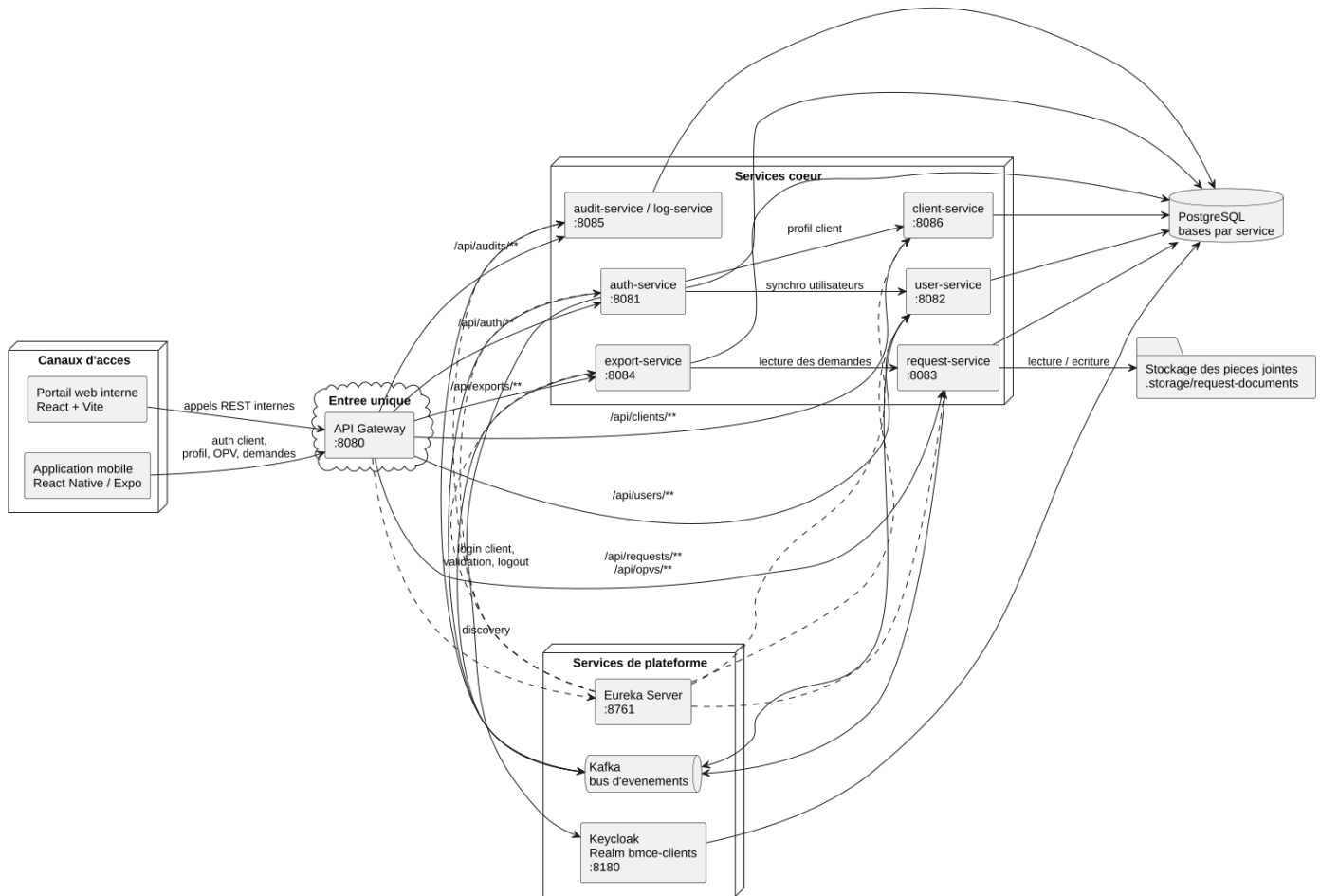


FIGURE 3.1 – Architecture technique globale de la solution web et mobile

3.2 Architecture fonctionnelle

La conception fonctionnelle repose sur trois rôles principaux côté portail web :

- **Agent** : prise en charge et traitement des demandes ;
- **Administrateur** : supervision opérationnelle, gestion des utilisateurs et actions transverses ;
- **Spectateur** : consultation et export des demandes finalisées.

Un quatrième acteur complète la solution sur le canal mobile :

- **Client** : consultation des OPV, authentification mobile et soumission d'une souscription.

Le cycle de vie principal d'une demande suit l'enchaînement **PENDING** → **IN_PROGRESS** → **APPROVED** ou **REJECTED**. Dans le cas mobile, la demande est liée à une OPV et peut comporter des pièces justificatives complémentaires.

3.3 Organisation technique du projet

Sur le plan technique, le projet est structuré en modules séparés, chacun correspondant à une responsabilité métier ou transverse clairement identifiée. Cette organisation facilite la maintenance, l'évolution de la plateforme et l'isolation des problèmes.

- **eureka-server** : enregistrement et découverte des services ;
- **api-gateway** : filtrage, sécurisation et routage des requêtes ;
- **auth-service** : authentification interne et gestion des JWT ;
- **user-service** : gestion des utilisateurs et des rôles ;
- **client-service** : exposition des profils clients et des données du parcours mobile ;
- **request-service** : traitement métier des demandes, gestion des OPV et des pièces jointes ;
- **export-service** : production des exports Excel ;
- **audit-service** : centralisation des journaux d'activité ;
- **frontend** : interface React destinée aux profils internes ;
- **BMCE-mobile-pfe** : application mobile React Native / Expo.

Un script unique **start.sh** a également été consolidé pour simplifier le démarrage de l'environnement, le packaging des services, les redémarrages ciblés et les vérifications d'état de la plateforme.

3.4 Outils et technologies

Les choix techniques ont été guidés par la volonté de construire une plateforme réaliste, modulaire, traçable et simple à faire évoluer.

- **Java 17** comme base de compilation et d'exécution backend ;
- **Spring Boot 3.2.3** pour la mise en oeuvre des microservices ;
- **Spring Cloud 2023.0.0** pour l'intégration distribuée ;
- **React 19**, **TypeScript**, **Vite** et **Tailwind CSS 4** pour le front-end web ;
- **React Native** et **Expo** pour l'application mobile ;
- **PostgreSQL 15** pour la persistance relationnelle ;
- **Apache Kafka** pour la propagation asynchrone des événements d'audit ;
- **Keycloak** pour l'authentification des clients mobiles ;
- **Apache POI** pour la génération des exports Excel ;
- **Docker Compose** pour l'environnement local de démonstration ;
- **Maven** pour le build des services backend ;
- **GitLab**, **Jenkins** et **SonarQube** pour l'industrialisation progressive de la chaîne CI/CD.

TABLE 3.1 – Technologies principales du projet

Catégorie	Technologie	Rôle dans le projet
Backend	Spring Boot 3.2.3	Développement des microservices
Backend	Java 17	Langage de développement et d'exécution
Cloud	Spring Cloud 2023.0.0	Découverte de services et intégration distribuée
Frontend	React 19	Construction de l'interface web
Frontend	TypeScript	Typage du front-end
Frontend	Vite	Build et environnement de développement
Frontend	Tailwind CSS 4	Mise en forme de l'interface
Mobile	React Native / Expo	Construction de l'application mobile
Données	PostgreSQL 15	Stockage relationnel
Messaging	Kafka / Zookeeper	Propagation des événements d'audit
IAM	Keycloak	Authentification des comptes clients mobiles
Export	Apache POI	Génération des fichiers Excel
Infra	Docker Compose	Démarrage de l'infrastructure locale
Build	Maven	Build du backend multi-modules
CI/CD	GitLab, Jenkins, SonarQube	Gestion de code, build et qualité logicielle

Cette figure regroupe les principales technologies utilisées pour construire la solution : backend Spring Boot, interfaces React et React Native, persistance PostgreSQL, messagerie Kafka et outillage de build.



FIGURE 3.2 – Principales technologies mobilisées dans le projet

3.5 Justification des choix techniques

Ce socle technique est cohérent avec les objectifs du projet. L'architecture microservices facilite la séparation des responsabilités, tandis que l'association React / React Native permet de couvrir les usages web et mobile avec des technologies modernes et maintenables. PostgreSQL répond aux besoins de persistance structurée, Kafka renforce la traçabilité événementielle, et Keycloak apporte une base solide pour l'authentification mobile.

Enfin, l'ajout progressif des outils DevOps prépare la solution à une montée en maturité vers des pratiques d'intégration continue, de contrôle qualité et de déploiement plus industrialisées.

3.6 Exemple d'outillage technique

L'un des apports techniques importants du projet concerne l'automatisation de l'environnement local. Le script de démarrage a été ajusté pour charger les variables d'environnement nécessaires, empaqueter les artefacts manquants, démarrer les services en mode détaché et faciliter les opérations de redémarrage ciblé.

Listing 3.1 – Exemples de commandes de pilotage de la plateforme

```
1 ./start.sh
2 ./start.sh --quick
3 ./start.sh --status
4 ./start.sh --rebuild-restart request-service
5 ./start.sh --stop
```

3.7 Analyse fonctionnelle

L'analyse fonctionnelle a permis de structurer le projet autour de deux canaux complémentaires : un portail web interne destiné aux profils métier et une application mobile destinée aux clients souhaitant consulter les OPV et soumettre une souscription. Cette phase a servi à identifier les acteurs, à préciser leurs responsabilités et à formaliser les principaux scénarios métier avant la phase de réalisation.

3.7.1 Acteurs et périmètre fonctionnel

Les acteurs retenus dans l'analyse sont les suivants :

- **Administrateur** : gère les utilisateurs, supervise le catalogue OPV, consulte les journaux d'audit et pilote les opérations transverses ;
- **Agent** : traite les demandes, prend en charge les dossiers et exécute les actions métier de validation ou de rejet ;
- **Spectateur** : consulte les demandes finalisées et déclenche les exports ;
- **Client mobile** : consulte les OPV, s'authentifie, saisit une souscription et suit son historique.

Le modèle fonctionnel fait apparaître une généralisation entre les trois acteurs internes : l'**Agent** reprend le périmètre de consultation du **Spectateur**, puis l'**Administrateur** spécialise ce périmètre avec les fonctions de supervision et d'administration. Cette relation sert à montrer la proximité des rôles internes, tout en gardant les actions métier séparées : l'agent traite les dossiers, l'administrateur supervise et administre, tandis que l'export reste explicitement réservé au spectateur. Le **Client mobile** reste indépendant, car il utilise un canal séparé.

Ce diagramme montre la généralisation entre les profils internes, distingue la consultation commune, l'export réservé au spectateur, les fonctions d'administration et de traitement, ainsi que le parcours indépendant du client mobile.

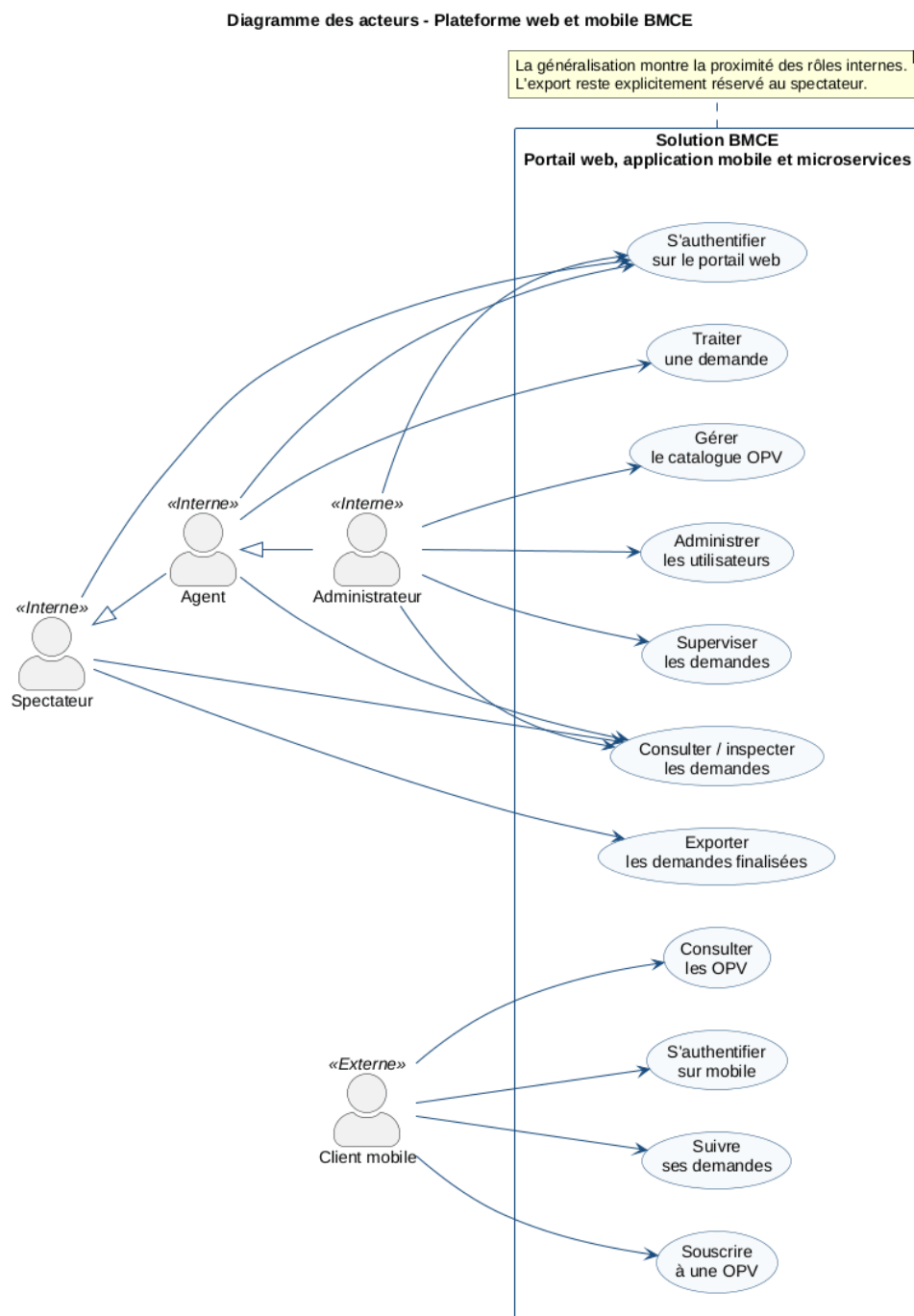


FIGURE 3.3 – Diagramme des acteurs de la solution

3.7.2 Diagrammes de cas d'utilisation

La modélisation des cas d'utilisation a été découpée par canal et par responsabilité afin de garder une lecture exploitable dans le mémoire. Les trois vues retenues couvrent l'administration, le traitement des demandes et le parcours mobile client.

Ce diagramme montre ce que l'administrateur peut faire dans le portail web. On y voit principalement les actions de supervision et de gestion : administrer les utilisateurs, consulter les demandes, réaffecter ou libérer un dossier, gérer le catalogue OPV et consulter les journaux d'audit.

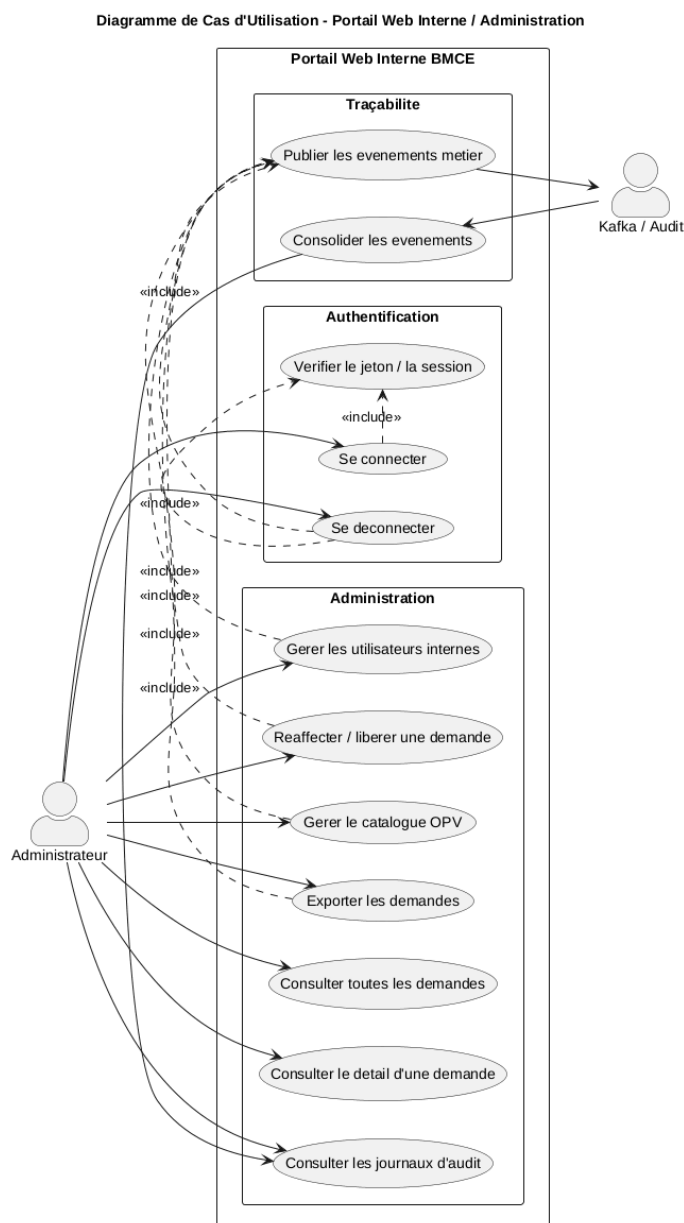


FIGURE 3.4 – Cas d'utilisation associés aux fonctions d'administration

Ce diagramme présente le travail quotidien des profils opérationnels. L'agent consulte ses demandes, prend en charge un dossier, puis décide de l'approuver ou de le rejeter, tandis que le spectateur intervient sur la consultation finale et l'export des demandes terminées.

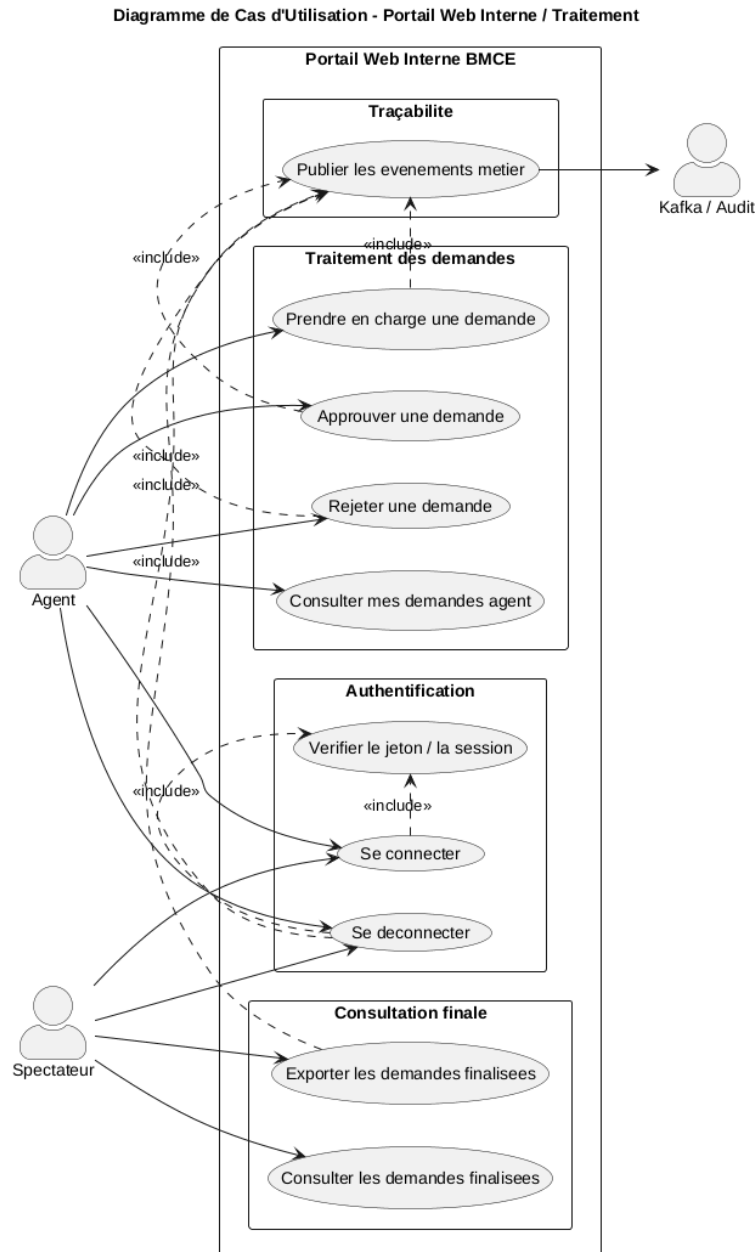


FIGURE 3.5 – Cas d'utilisation associés au traitement des demandes

Ce diagramme résume le parcours du client sur mobile. Il montre que le client s'authentifie, consulte les OPV disponibles, saisit une souscription, ajoute si besoin les pièces justificatives, puis suit l'état de ses demandes déjà envoyées.

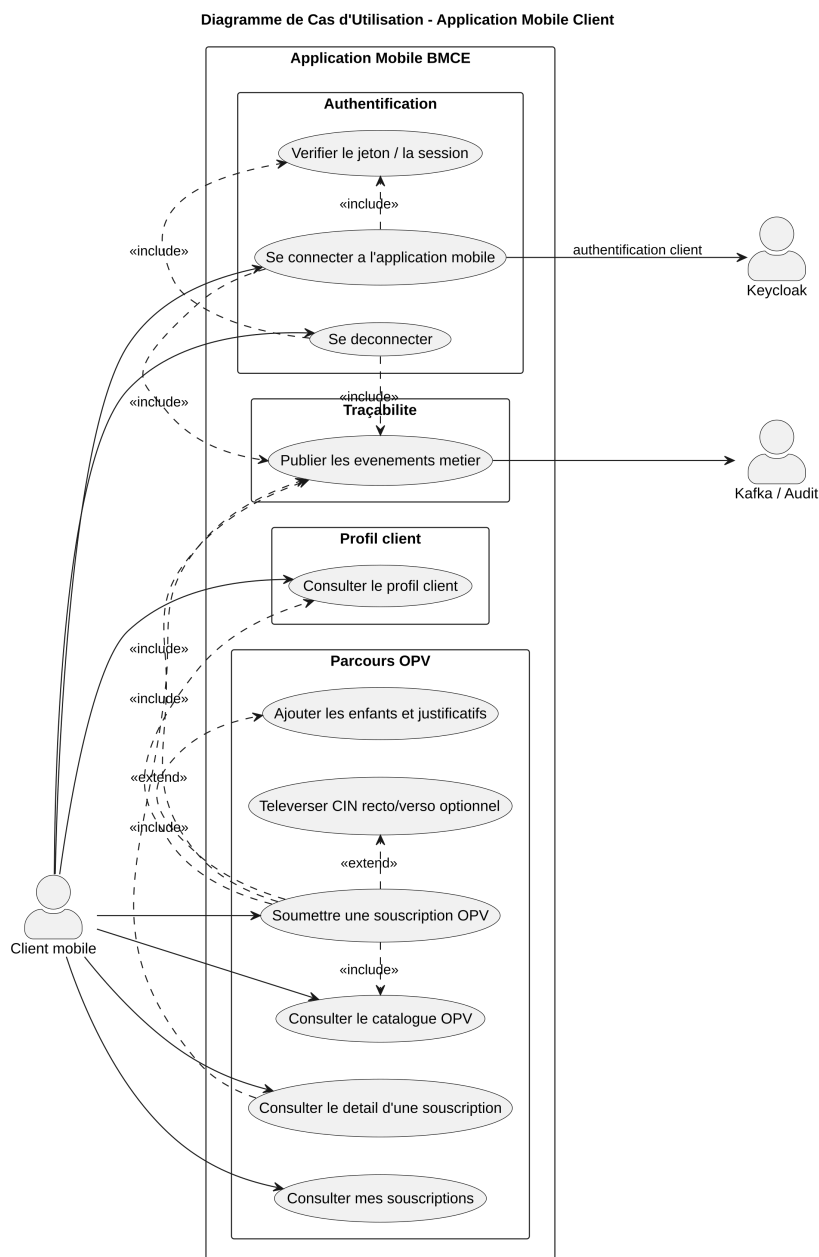


FIGURE 3.6 – Cas d'utilisation de l'application mobile client

3.8 Conception

La phase de conception a été structurée autour de trois vues complémentaires :

- Une **vue statique** pour décrire les entités et les relations principales ;
- Une **vue dynamique** pour expliciter les enchaînements d'appels entre les composants ;
- Une **vue de navigation** pour représenter les parcours offerts sur le web et sur mobile.

3.8.1 Conception statique

La conception statique a été reformulée dans le rapport sous forme de vues de domaine plus compactes, afin de préserver la lisibilité lors d'une impression A4. Les diagrammes suivants mettent en évidence, d'une part, la gestion des identités et des profils et, d'autre part, le coeur métier des demandes et des OPV.

Cette vue statique représente les éléments liés à l'identité et aux rôles. Elle permet de comprendre les informations portées par les comptes internes et les profils clients, ainsi que la séparation entre les utilisateurs du portail interne et les clients mobiles.

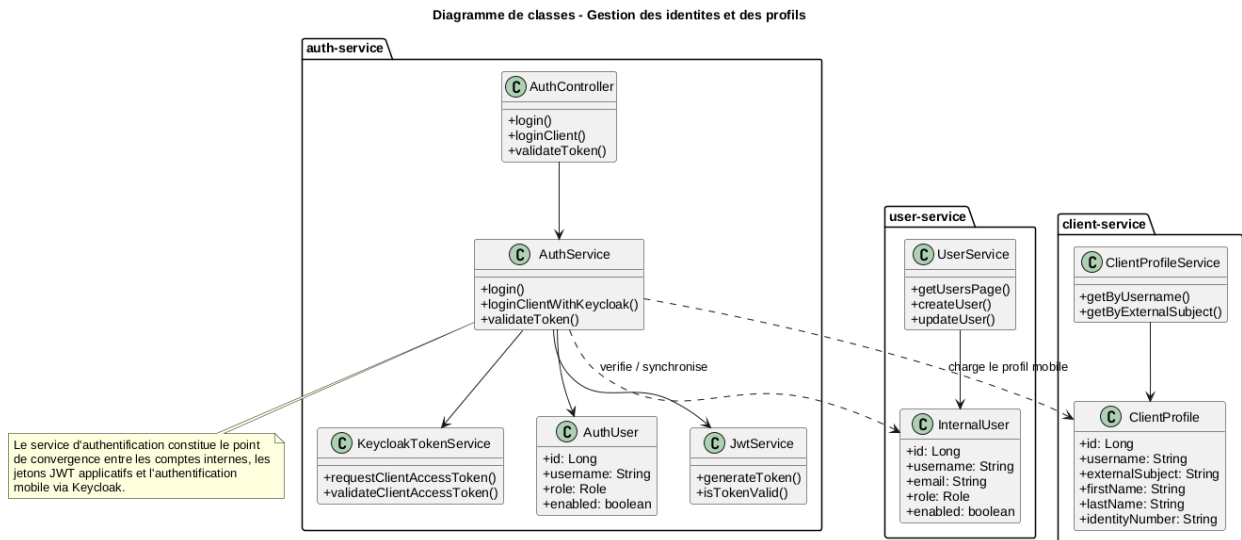


FIGURE 3.7 – Vue statique de la gestion des identités et des profils

Ce diagramme montre les objets métier centraux du projet. Il relie la demande de souscription, l'OPV concernée, les informations financières, les pièces jointes et les états du dossier afin d'expliquer le cycle de vie d'une demande.

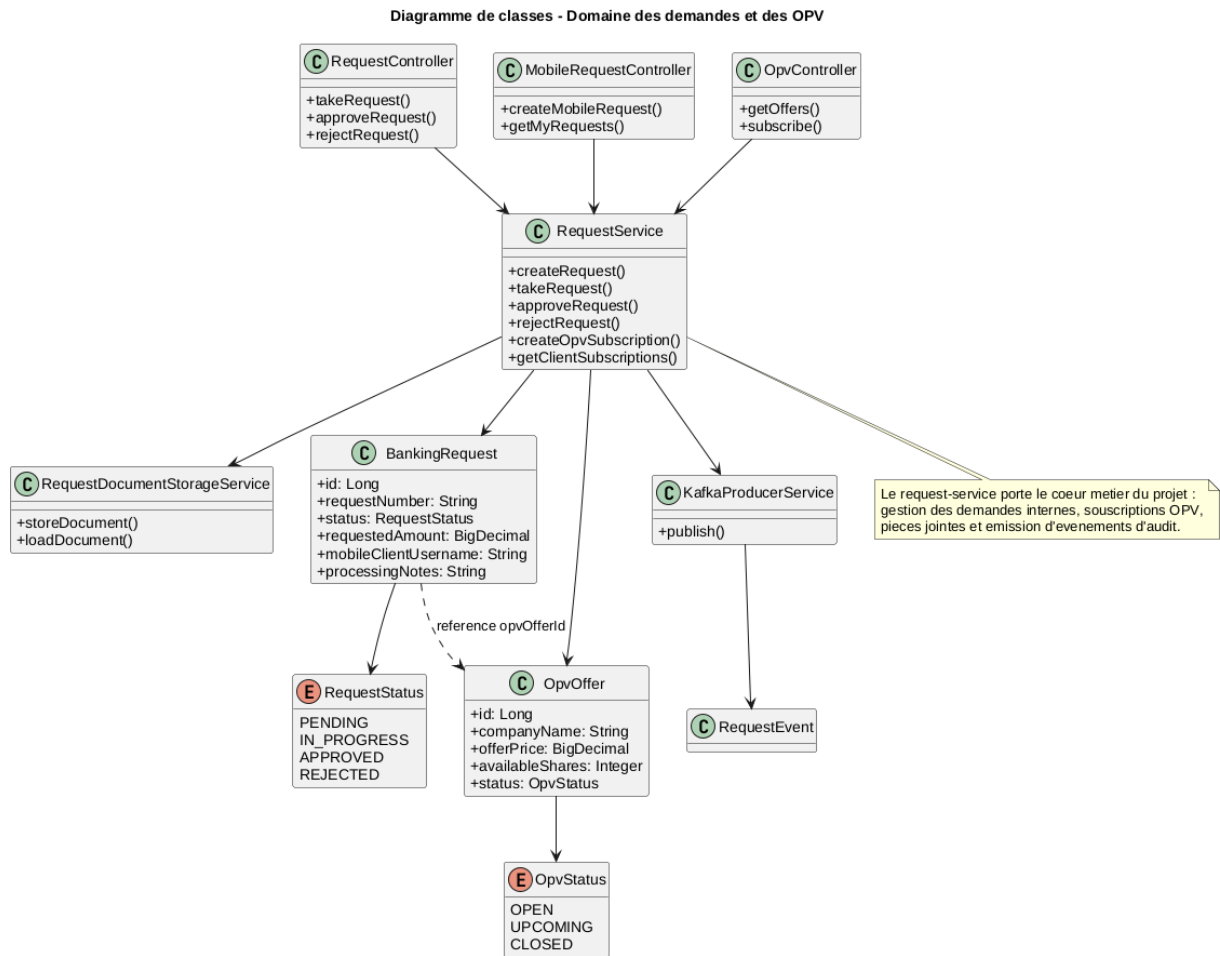


FIGURE 3.8 – Vue statique du domaine des demandes et des OPV

Cette vue isole les services techniques qui accompagnent le coeur métier. Elle montre comment l'audit centralise les événements produits par la plateforme et comment l'export exploite les dossiers finalisés pour générer les fichiers de restitution.

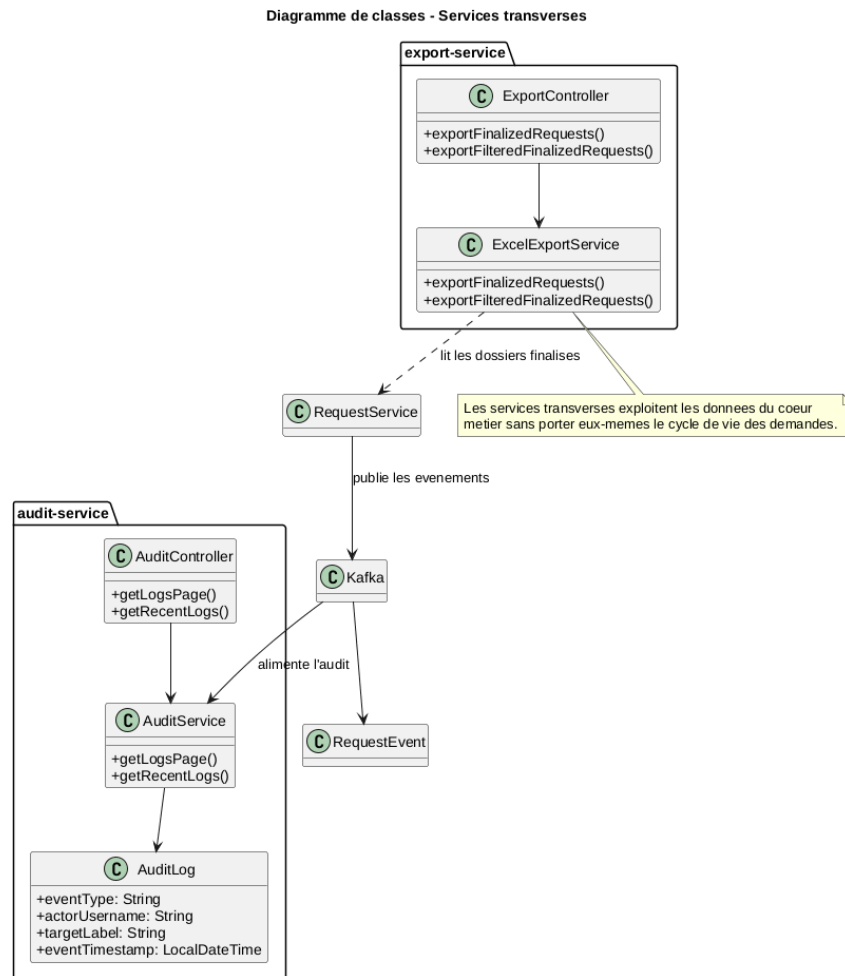


FIGURE 3.9 – Vue statique des services transverses d'audit et d'export

3.8.2 Conception dynamique

Les diagrammes de séquence ont été simplifiés pour faire ressortir la logique fonctionnelle des scénarios critiques sans charger inutilement la lecture. Cette version est plus adaptée à un rapport imprimé, car elle conserve les échanges importants tout en limitant le niveau de détail purement technique.

Ce diagramme de séquence décrit l'authentification d'un utilisateur interne depuis le portail web. Le front envoie les identifiants, les services vérifient l'identité, retournent le jeton puis enregistrent l'événement d'authentification.

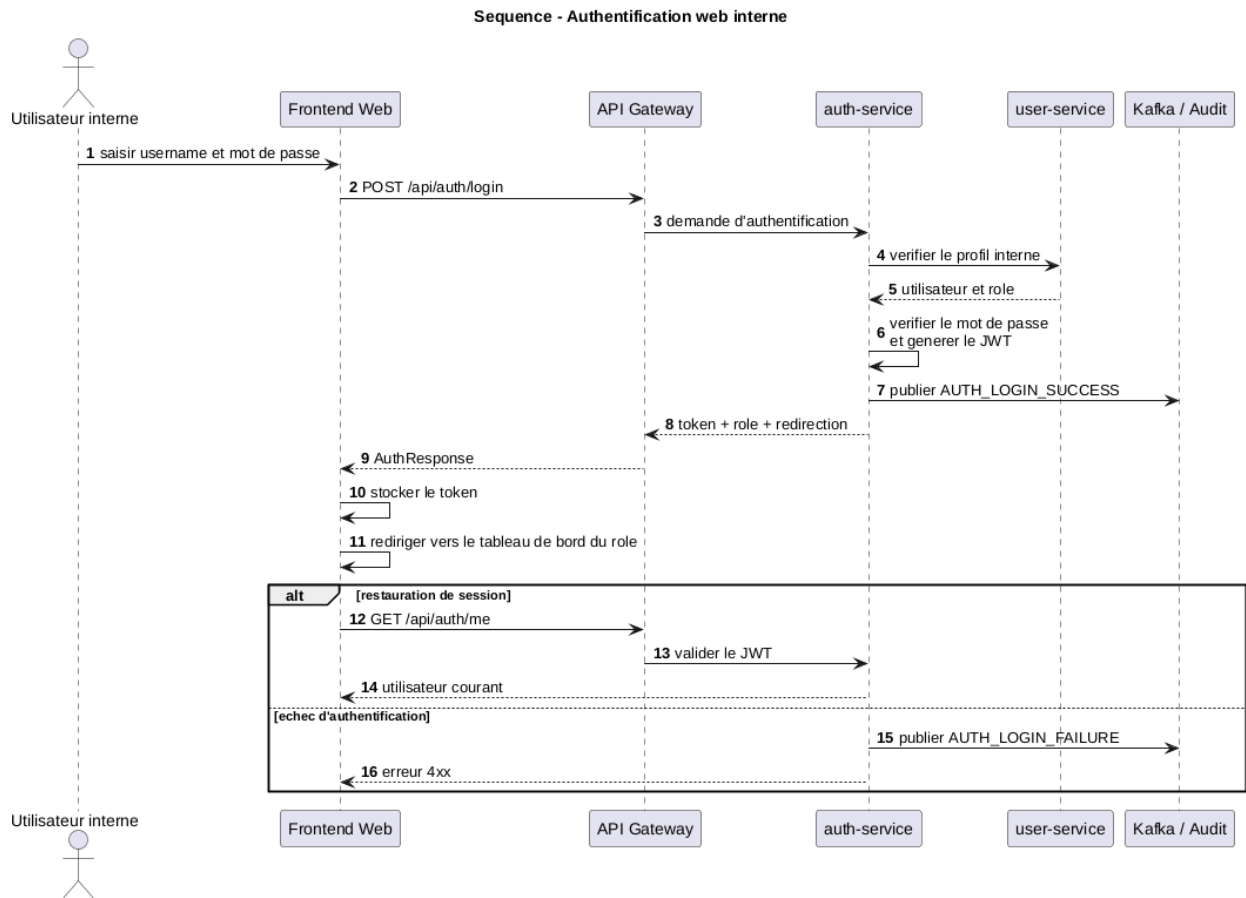


FIGURE 3.10 – Séquence d'authentification sur le portail web interne

Cette séquence décrit la connexion du client mobile avec Keycloak. Elle montre comment l'application transmet les identifiants, récupère une session valide, puis réutilise ce contexte pour consulter les services protégés.

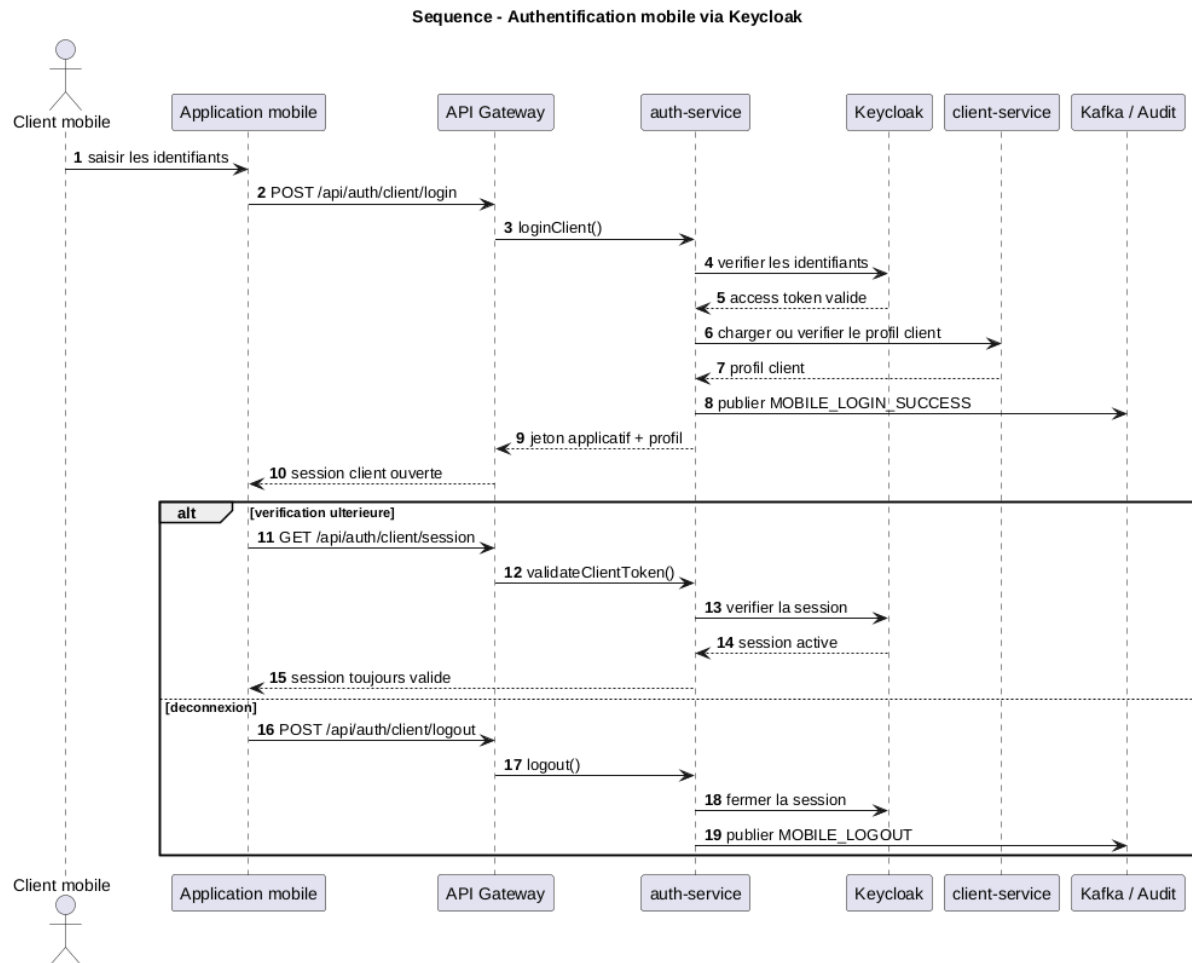


FIGURE 3.11 – Séquence d'authentification mobile via Keycloak

Cette séquence explique comment une souscription OPV est créée depuis le mobile : saisie du formulaire, vérification des données du client, enregistrement des pièces justificatives et création finale de la demande dans le backend.

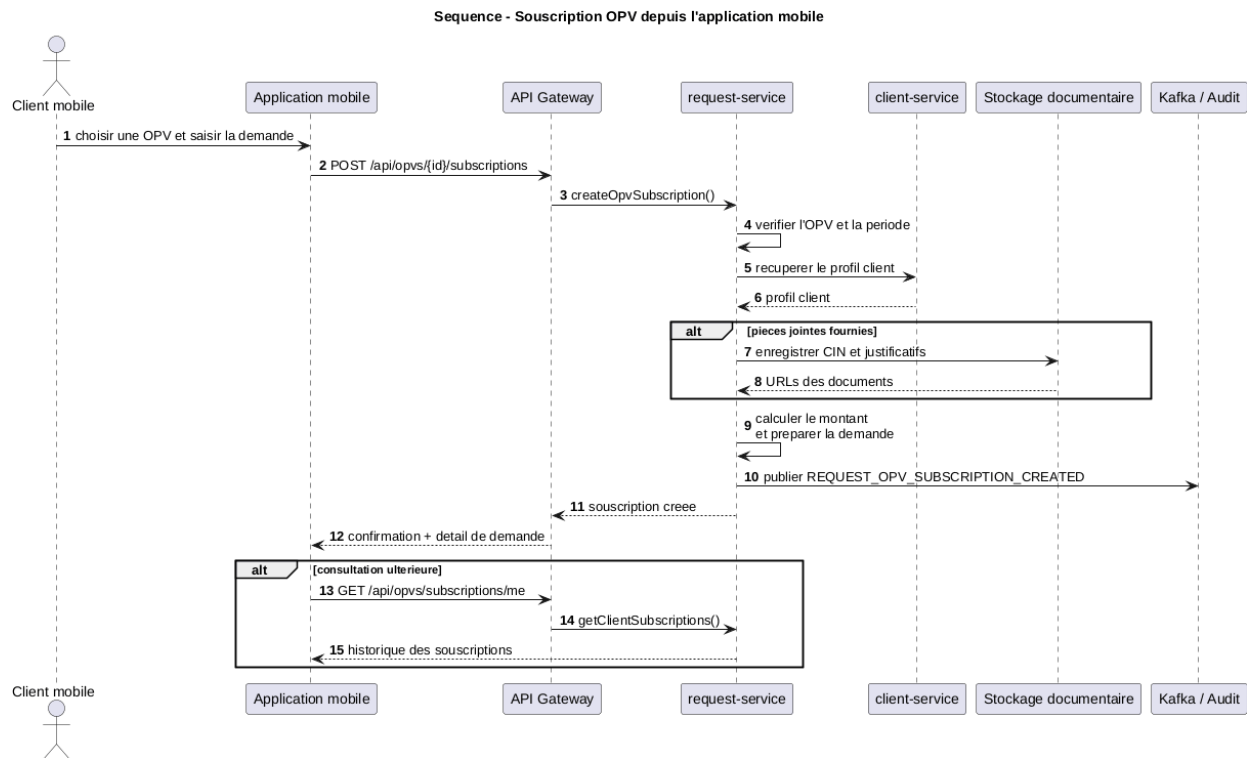


FIGURE 3.12 – Séquence de souscription d'une OPV depuis l'application mobile

Ce diagramme décrit le traitement d'un dossier côté portail interne. Il montre comment un agent consulte la liste des demandes, prend un dossier en charge, applique une décision puis déclenche la mise à jour de l'état et la publication de l'événement associé.

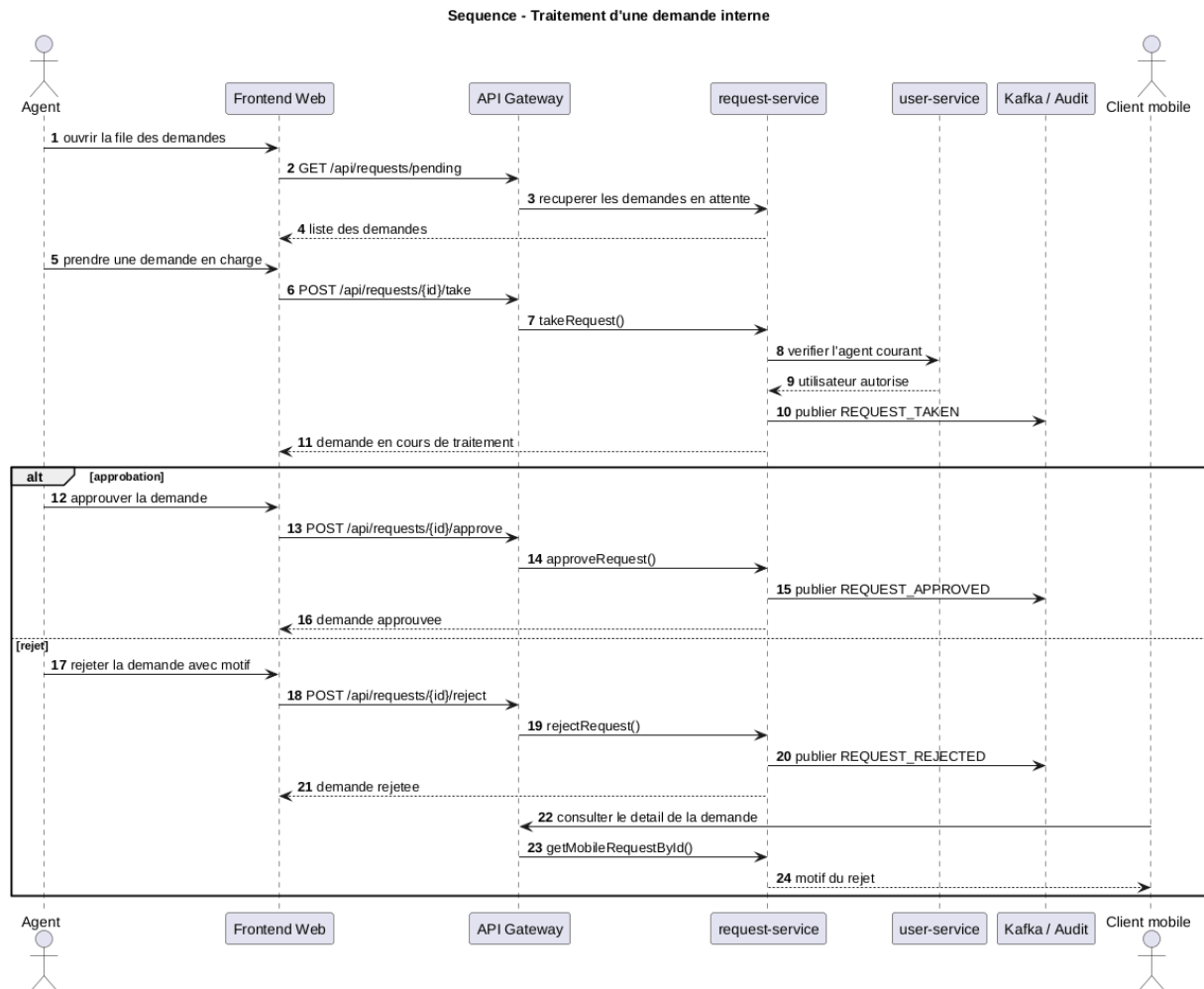


FIGURE 3.13 – Séquence de prise en charge et de traitement d'une demande

3.8.3 Conception de la navigation

La navigation a été modélisée séparément pour le portail web et pour l'application mobile. Les parcours web sont divisés entre la partie administration et les parcours opérationnels. Les parcours mobiles sont présentés entre la navigation générale de l'application et le sous-parcours dédié à la souscription OPV.

Ce schéma se lit comme un plan des écrans disponibles pour l'administrateur. Il montre les pages principales du portail et les chemins possibles pour gérer les utilisateurs, les demandes, le catalogue OPV et l'audit.

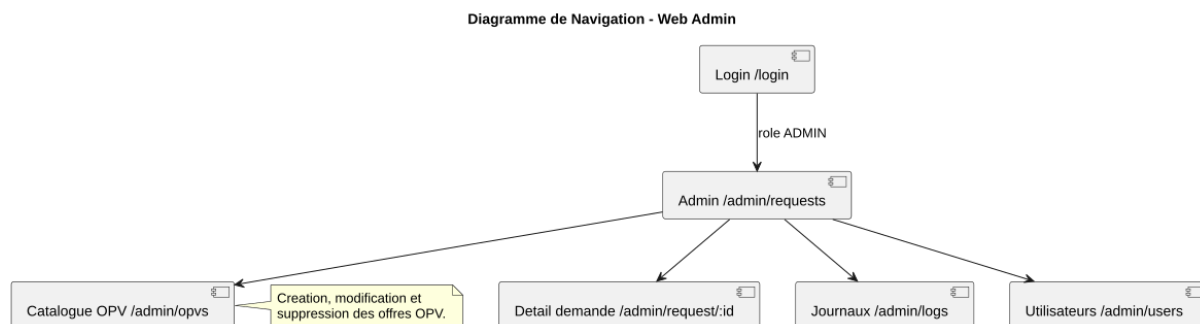


FIGURE 3.14 – Navigation du portail web côté administration

Cette vue complète la précédente en se concentrant sur les parcours métier du portail. Elle met en évidence les écrans parcourus par l'agent et le spectateur pour consulter, traiter ou exporter les demandes selon leur rôle.

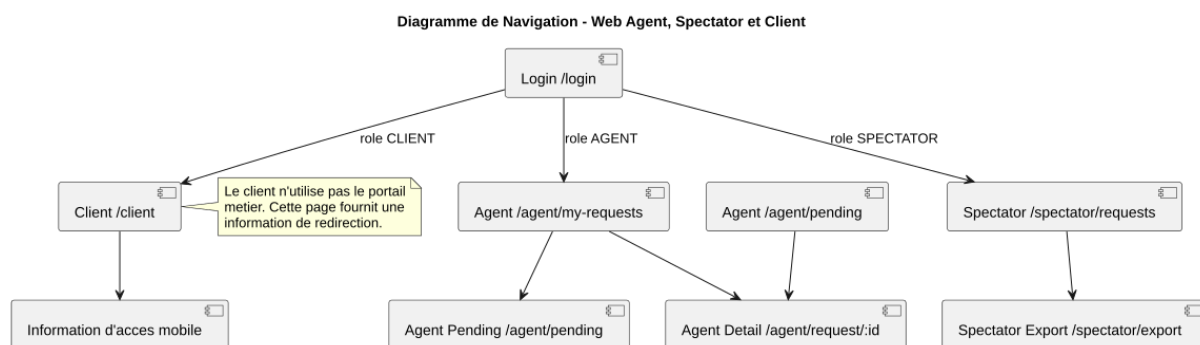


FIGURE 3.15 – Navigation du portail web pour les parcours agent, spectateur et consultation métier

Ce diagramme présente la structure générale de navigation mobile. Il aide à comprendre comment l'utilisateur passe d'un onglet principal à un autre à partir de l'écran d'accueil de l'application.

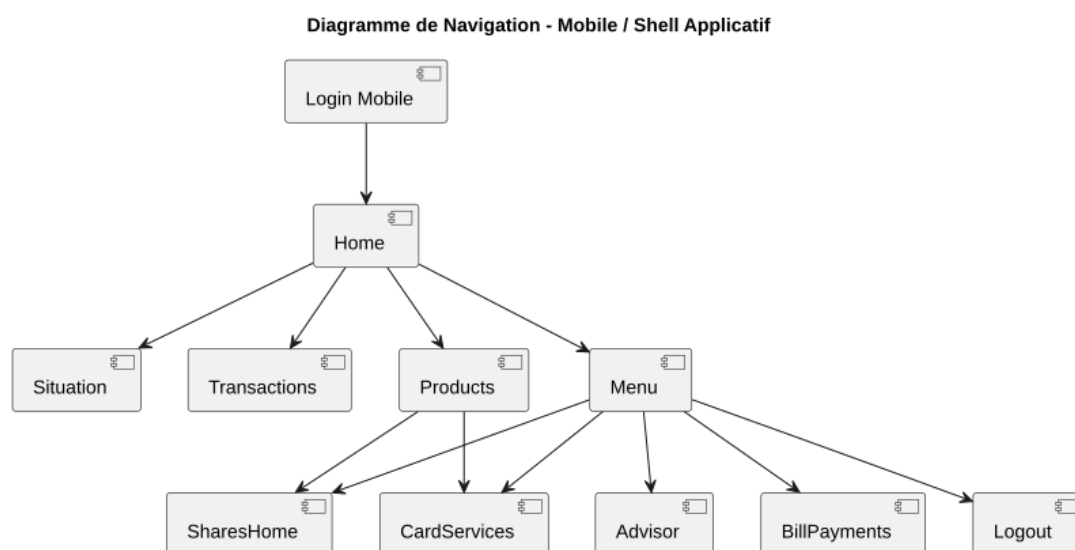


FIGURE 3.16 – Shell de navigation de l'application mobile

Ce dernier schéma zoome sur le parcours mobile de souscription. Il montre les écrans traversés par le client depuis la création d'une demande jusqu'à la consultation du résultat et des détails enregistrés.

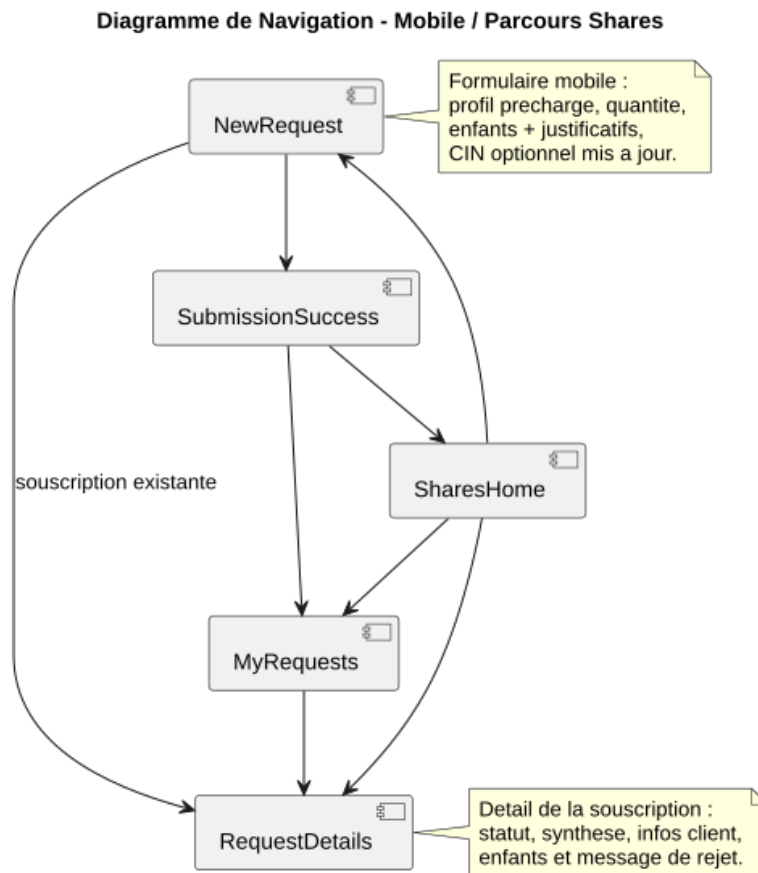


FIGURE 3.17 – Parcours mobile dédié à la souscription OPV

Chapitre 4

Réalisation du système

4.1 Mise en oeuvre fonctionnelle

Ce chapitre présente la manière dont les fonctionnalités ont été concrètement implémentées dans la solution, aussi bien côté portail web que côté application mobile. L'objectif est de montrer la traduction opérationnelle du besoin métier dans les parcours utilisateurs effectivement réalisés.

4.2 Fonctionnalités principales

4.2.1 Authentification et rôles

L'utilisateur interne se connecte via un compte web. Le système applique ensuite des droits différents selon le rôle :

- **ADMIN** : supervision des demandes en cours et gestion des utilisateurs ;
- **AGENT** : prise en charge et traitement des demandes ;
- **SPECTATOR** : consultation et export des demandes finalisées.

Le parcours mobile repose sur un compte **CLIENT**. L'authentification du client ne passe pas par le formulaire interne classique, mais par un échange avec Keycloak, suivi de l'émission d'un jeton applicatif BMCE utilisé par les microservices.

4.2.2 Gestion du workflow métier

Le coeur du projet est le cycle de vie de la demande. Une demande peut être créée ou soumise, prise en charge par un agent, approuvée ou rejetée. Le rejet impose une note explicative. Les administrateurs peuvent également libérer une demande ou la réaffecter à un autre agent. Dans le parcours mobile, la demande est rattachée à une OPV et peut comporter des pièces justificatives, notamment les documents des enfants et, si nécessaire, une mise à jour optionnelle des images de CIN.

Dans le périmètre mobile, la logique OPV ne se réduit pas à un simple envoi de formulaire.

Le client consulte un catalogue structuré en offres ouvertes, à venir et clôturées, puis ne peut engager une souscription que sur une offre ouverte. La demande générée intègre la quantité d'actions demandée, les informations client préchargées depuis le `client-service`, les pièces complémentaires liées aux enfants et, si nécessaire, une mise à jour optionnelle des images de CIN.

4.2.3 Consultation et filtrage

Les écrans principaux reposent sur une pagination et des filtres côté serveur : recherche, dates, statut, rôle utilisateur. Cette approche évite de charger inutilement de grands volumes côté client et garantit un comportement cohérent sur toutes les vues critiques. L'application mobile applique la même logique pour les listes importantes, notamment les souscriptions du client et le catalogue OPV classé par état : ouverte, à venir, clôturée.

4.2.4 Export des résultats

Les demandes finalisées peuvent être exportées en Excel selon plusieurs modes : export complet, export filtré et export sur sélection explicite. Le service d'export a été revu pour éviter un parcours inefficace consistant à charger la totalité des données finalisées avant de les filtrer localement.

4.2.5 Audit et supervision

Les actions importantes sont consignées dans un journal d'activité. Les événements d'authentification, de traitement des demandes, de gestion des utilisateurs, d'export et de souscription OPV sont centralisés afin d'améliorer la traçabilité.

4.2.6 Durcissement sécurité et fiabilité

Une passe de revue a permis de corriger plusieurs faiblesses concrètes : restrictions d'accès, synchronisation inter-services des utilisateurs, protection des endpoints internes, amélioration des flux de réaffectation, gestion des sessions clients mobiles, contrôle d'appartenance sur les demandes mobiles et suppression de mécanismes de démonstration trop permissifs par défaut.

4.2.7 Démarche de réalisation

Le développement a été conduit de manière incrémentale. Une première étape a permis de stabiliser le backend et le portail web interne autour des rôles métier et du workflow principal. Une deuxième étape a introduit le catalogue OPV et le parcours mobile client. Une troisième étape a porté sur l'intégration de Keycloak, l'amélioration de la traçabilité, la consolidation documentaire et l'automatisation du démarrage de l'environnement local.

4.2.8 Scénario de référence retenu

Pour illustrer la réalisation sans multiplier les cas particuliers, le scénario de référence retenu est celui d'une souscription OPV créée côté mobile puis traitée côté portail web. Ce scénario concentre l'essentiel des choix réalisés dans le projet :

- Authentification du client via Keycloak ;
- Consultation du catalogue OPV ;
- Création d'une souscription avec pièces justificatives ;
- Prise en charge du dossier par un agent ;
- Décision métier et traçabilité via l'audit ;
- Consultation finale ou export selon le rôle utilisateur.

Ce scénario est pertinent pour le rapport car il montre en une seule chaîne fonctionnelle l'interaction entre le mobile, le portail web, les microservices métier, l'authentification, la journalisation et le workflow de traitement.

4.2.9 Répartition de l'implémentation

La réalisation a été répartie entre plusieurs composants spécialisés afin de conserver une architecture claire et maintenable. Le tableau suivant résume cette répartition et montre que chaque brique porte une responsabilité bien délimitée.

TABLE 4.1 – Répartition des responsabilités dans l'implémentation

Composant	Canal principal	Responsabilité portée
frontend	Web interne	Interfaces ADMIN , AGENT et SPECTATOR
mobile	Mobile client	Consultation OPV, souscription et suivi client
api-gateway	Web / Mobile	Routage centralisé et exposition des APIs
auth-service	Web / Mobile	Authentification, rôles, jetons JWT et pont avec Keycloak
user-service	Web interne	Gestion des utilisateurs internes
client-service	Mobile client	Profils clients, catalogue OPV et données de souscription
request-service	Web / Mobile	Cycle de vie des demandes et pièces justificatives
audit-service	Web / Mobile	Journalisation des actions critiques
export-service	Web interne	Génération des exports Excel

Cette répartition a facilité le développement incrémental. Les corrections fonctionnelles et de sécurité ont pu être appliquées au service concerné sans remettre en cause toute l'application, ce qui a été particulièrement utile pendant la phase de stabilisation finale.

4.3 Méthode de validation

La validation a été menée sur l’environnement local complet de démonstration, puis consolidée par des vérifications ciblées sur les parcours critiques. L’objectif n’était pas de multiplier les captures, mais de démontrer qu’un scénario représentatif fonctionne de bout en bout avec les rôles et les services attendus.

La démarche de validation a combiné :

- La compilation du backend via Maven ;
- Le build de production du front-end React ;
- La compilation locale des diagrammes PlantUML utilisés dans la documentation ;
- Les tests unitaires et d’intégration disponibles dans les services ;
- Les tests manuels des parcours ADMIN, AGENT, SPECTATOR et CLIENT ;
- La vérification du bon démarrage des services et de leur enregistrement via Eureka.

4.4 Scénario fonctionnel de référence

Le scénario retenu pour illustrer la réalisation est celui d’une souscription OPV créée par un client mobile, prise en charge par un agent, puis rendue visible dans les vues de supervision et de consultation. Ce choix permet de montrer une seule chaîne complète au lieu de juxtaposer un grand nombre de captures redondantes.

4.4.1 Étape 1 : création de la souscription côté mobile

Le client s’authentifie dans l’application mobile, consulte le catalogue OPV, puis démarre une souscription sur une offre ouverte. Il saisit la quantité demandée, complète les informations nécessaires et joint les pièces justificatives attendues.

Cette étape valide que le mobile ne se limite pas à un simple affichage de données : il orchestre un vrai parcours métier, sécurisé par Keycloak et relié aux services backend du projet.

4.4.2 Étape 2 : prise en charge et traitement côté agent

Une fois la demande créée, elle apparaît dans la file d’attente du portail web. L’agent peut alors consulter le dossier, analyser les pièces, lire les informations métier et appliquer une décision conforme au workflow.

Cette étape montre que le portail web prend le relais du mobile dans un cadre de travail plus riche, avec séparation des responsabilités et contrôle explicite du cycle de vie de la demande.

Cette série de captures montre le parcours mobile retenu pour la démonstration : connexion du client, choix de l'OPV, saisie de la souscription et finalisation de la demande.

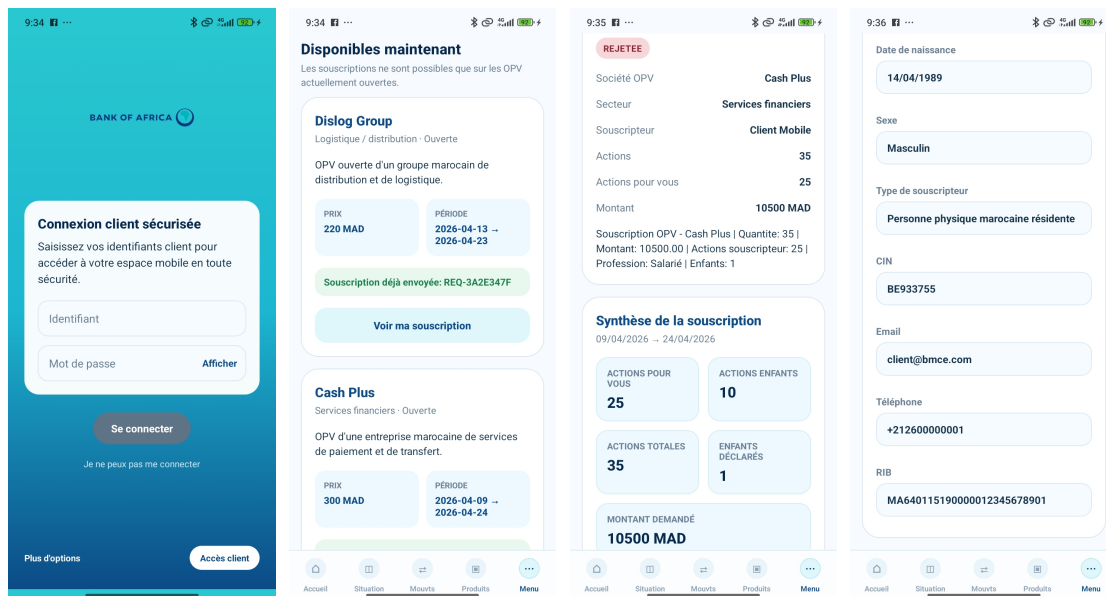


FIGURE 4.1 – Souscription OPV côté mobile : authentification, choix de l'offre, saisie et finalisation

Cette série de captures présente le relais côté agent : consultation de la file d'attente, ouverture du dossier, lecture des documents et vérification des informations métier.

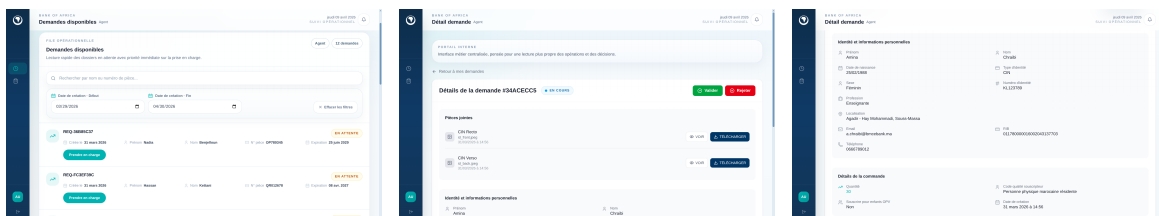


FIGURE 4.2 – Traitement du dossier côté agent : file d'attente, pièces justificatives et détails métier

4.4.3 Étape 3 : supervision et consultation finale

Après traitement, la demande devient visible selon le rôle. L'administrateur supervise les opérations et les comptes, tandis que le spectateur consulte les dossiers finalisés et déclenche l'export lorsque cela est nécessaire.

Cette série de captures illustre la supervision finale : suivi administratif des demandes, consultation des journaux d'audit et export des dossiers finalisés.



FIGURE 4.3 – Fin de chaîne : supervision administrative, audit et consultation/export des dossiers finalisés

L'ensemble du scénario confirme que la plateforme couvre bien la continuité entre le canal client mobile, le portail web interne et les fonctions transverses de traçabilité et d'export.

4.5 Synthèse des résultats

Les résultats fonctionnels les plus importants sont les suivants :

- Mise en place d'une architecture distribuée cohérente pour le portail web et l'application mobile ;
- Séparation claire des rôles **ADMIN**, **AGENT**, **SPECTATOR** et **CLIENT** ;
- Intégration d'une authentification mobile client via Keycloak ;
- Mise en place d'un catalogue OPV administrable côté web et consommable côté mobile ;
- Disponibilité d'un workflow complet de demande, depuis la création jusqu'à la décision finale ;
- Traçabilité améliorée grâce au service d'audit ;
- Disponibilité d'un export Excel des dossiers finalisés.

TABLE 4.2 – Synthèse des validations réalisées

Élément validé	Méthode	Résultat	Preuve
Compilation backend	Maven	Conforme	Sortie de build
Build front-end	Vite / TypeScript	Conforme	Artefact web généré
Compilation des diagrammes	PlantUML local	Conforme	Fichiers PNG/SVG générés
Cycle de vie des demandes	Tests manuels	Conforme	Scénario web/mobile validé
Gestion des rôles	Tests fonctionnels	Conforme	Restrictions par profil
Authentification mobile client	Keycloak + tests manuels	Conforme	Session active côté mobile
Souscription OPV mobile	Tests fonctionnels	Conforme	Création et suivi d'une demande
Export Excel	Vérification fonctionnelle	Conforme	Export généré
Journalisation des actions	Consultation des logs	Conforme	Vue d'audit alimentée
Démarrage simplifié de la plateforme	Script <code>start.sh</code>	Conforme	Services enregistrés dans Eureka

Conclusion générale

Bilan

Ce projet m’a permis de concevoir et de réaliser une solution complète pour la gestion et la souscription d’actions, en combinant un portail web interne et une application mobile client. Le travail mené a couvert à la fois la modélisation du besoin, le développement des fonctionnalités métier, l’intégration de l’authentification, la traçabilité des opérations et la mise en place d’une première chaîne DevOps.

Le bilan principal est la mise à disposition d’une plateforme cohérente, exécutable et démontrable. Le portail web permet la gestion des utilisateurs, le traitement des demandes, la supervision, l’audit et l’export. L’application mobile permet au client de consulter les OPV et de soumettre une souscription reliée au même backend. L’ensemble repose sur une architecture microservices qui reste lisible et évolutive dans le cadre du projet.

Perspectives

Les perspectives naturelles du projet sont les suivantes :

- Renforcer la couverture de tests automatisés sur le backend, le web et le mobile ;
- Approfondir l’observabilité applicative avec davantage de métriques métier et techniques ;
- Poursuivre l’industrialisation du déploiement sur les environnements **recette** et **prod** ;
- Faire évoluer la gestion des secrets, du durcissement réseau et des accès d’administration ;
- Préparer une intégration plus poussée avec des briques d’entreprise telles que LDAP, bastion ou SIEM.

Ces perspectives prolongent directement le travail réalisé : partir d’une solution fonctionnelle de PFE pour la rapprocher progressivement d’un système plus robuste, mieux supervisé et plus proche des contraintes réelles d’exploitation.

Bibliographie

- [1] Eurafric Information. *Site officiel Eurafric Information*. <https://www.eurafric-information.com/>. Consulté en avril 2026.
- [2] Eurafric Information. *Présentation de l'entreprise*. <https://www.eurafric-information.com/fr/entreprise>. Consulté en avril 2026.
- [3] VMware. *Spring Boot Reference Documentation*. <https://docs.spring.io/spring-boot/docs/3.2.3/reference/html/>. Consulté en avril 2026.
- [4] VMware. *Spring Cloud Reference Documentation*. <https://docs.spring.io/spring-cloud/docs/2023.0.0/reference/html/>. Consulté en avril 2026.
- [5] Meta. *React Documentation*. <https://react.dev/>. Consulté en avril 2026.
- [6] PostgreSQL Global Development Group. *PostgreSQL Documentation*. <https://www.postgresql.org/docs/>. Consulté en avril 2026.
- [7] Apache Software Foundation. *Apache Kafka Documentation*. <https://kafka.apache.org/documentation/>. Consulté en avril 2026.
- [8] Keycloak. *Keycloak Documentation*. <https://www.keycloak.org/documentation>. Consulté en avril 2026.
- [9] Apache Software Foundation. *Apache POI Documentation*. <https://poi.apache.org/>. Consulté en avril 2026.

Annexe A

Industrialisation et environnement DevOps

A.1 Objectif de la phase DevOps

Après la stabilisation du portail web interne, de l'application mobile client et des microservices métier, une phase DevOps a été engagée pour industrialiser la solution. L'objectif n'était plus seulement de faire fonctionner l'application en local, mais de disposer d'une chaîne crédible de build, d'analyse, de publication et de déploiement sur des environnements séparés.

Cette phase a visé principalement à :

- Mettre en place une base d'hébergement dédiée sous Proxmox VE ;
- Isoler les services d'usine logicielle dans des VMs spécialisées ;
- Préparer les environnements `dev`, `recette` et `prod` ;
- Automatiser la chaîne `GitLab` → `Jenkins` → `SonarQube/Nexus/Harbor` → `Kubernetes` ;
- Ajouter une supervision simple mais exploitable via Prometheus et Grafana.

A.2 Architecture cible de l'usine DevOps

L'architecture retenue repose sur un hyperviseur Proxmox VE et sur un ensemble de VMs dédiées aux briques principales de la chaîne DevOps. Ce choix permet d'isoler les rôles, de limiter les conflits de dépendances et de garder une structure démontrable dans un contexte de PFE.

La figure suivante synthétise les briques logicielles les plus importantes de cette usine.

A.3 Socle d'hébergement et réseau

Le laboratoire a été construit sur un sous-réseau `192.168.11.0/24`. L'hyperviseur Proxmox a été stabilisé sur `192.168.11.10` avec un bridge `vmbbr0`. Une difficulté de démarrage liée au noyau Proxmox et au module `nvidiafb` a été rencontrée au début du projet, puis corrigée

TABLE A.1 – Composants de l'usine DevOps

Composant	Rôle	Usage dans le projet
Proxmox VE	Hyperviseur	Hébergement des VMs de plateforme
GitLab	Forge logicielle	Dépôts Git, branches, droits et structure projet
Jenkins	CI/CD	Build, tests, analyse qualité et déploiement
SonarQube	Qualité logicielle	Analyse statique et <i>quality gates</i>
Nexus	Dépôt d'artefacts	Proxy Maven et publication des jars backend
Harbor	Registry d'images	Publication des images Docker applicatives
Kubernetes (k3s)	Orchestration	Cible de déploiement des environnements
Prometheus / Grafana	Supervision	Métriques et tableaux de bord de plateforme
Tailscale	Accès distant sécurisé	Administration distante du laboratoire

Cette figure regroupe les briques de l'usine DevOps mise en place autour du projet : virtualisation, gestion de code, intégration continue, qualité, artefacts, images, déploiement et supervision.



FIGURE A.1 – Principales briques de l'usine DevOps du projet

par désactivation des modules graphiques problématiques et ajustement du démarrage du système.

Cette étape a été importante car elle a transformé une machine hôte instable en base d'hébergement fiable pour toutes les VMs de plateforme. L'accès distant a ensuite été sécurisé avec Tailscale afin de permettre l'administration hors du réseau local sans exposition directe sur Internet.

A.4 Briques effectivement mises en service

La plateforme finale a été construite autour des VMs suivantes :

TABLE A.2 – VMs principales du laboratoire DevOps

VM	Adresse	Ressources	Rôle principal
GitLab	192.168.11.20	Ubuntu Server	Forge logicielle
Jenkins	192.168.11.30	3 vCPU / 6 Go RAM	Orchestration CI/CD
SonarQube	192.168.11.40	3 vCPU / 5 Go RAM	Analyse qualité
Harbor	192.168.11.50	2 vCPU / 4 Go RAM	Registry d'images
Nexus	192.168.11.60	2 vCPU / 4 Go RAM	Dépôt d'artefacts Maven
k8s-single	192.168.11.70	4 vCPU / 8 Go RAM	Cluster k3s mono-noeud
Monitoring	192.168.11.80	2 vCPU / 4 Go RAM	Prometheus + Grafana

L'intérêt de cette répartition est double :

- Chaque brique garde un rôle clairement identifiable dans la chaîne ;
- Les incidents restent plus faciles à diagnostiquer qu'avec une installation monolithique sur une seule VM.

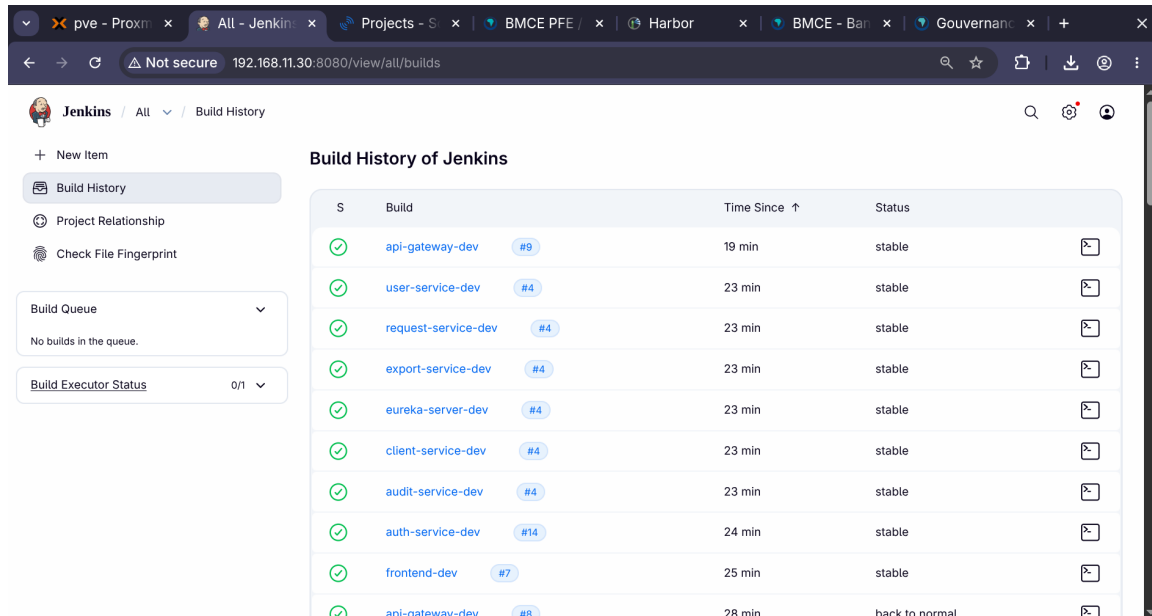
A.5 Chaîne CI/CD mise en oeuvre

La chaîne CI/CD réellement opérationnelle pendant le projet repose sur les étapes suivantes :

1. Le code source est versionné dans GitLab ;
2. Jenkins exécute un `Jenkinsfile` mutualisé pour les services ;
3. Maven construit les modules backend et publie les analyses SonarQube ;
4. Les jars backend sont publiés dans Nexus ;
5. Les images Docker sont construites puis publiées dans Harbor ;
6. Les services sont redéployés sur le cluster Kubernetes.

Le pipeline a été factorisé afin qu'un job nommé `<service>-<environnement>` déduise automatiquement le module Maven à traiter. Cette approche a évité la duplication d'un pipeline par microservice.

Cette capture montre les jobs Jenkins validés pour les services backend, utilisés pour compiler, analyser, publier et déployer les composants applicatifs.



The screenshot shows the Jenkins web interface at the 'Build History' page. The browser tabs include 'pve - Proxmox', 'All - Jenkins', 'Projects - S...', 'BMCE PFE', 'Harbor', 'BMCE - Bar', and 'Gouvernan...'. The address bar shows '192.168.11.30:8080/view/all/builds'. The Jenkins logo and 'All / Build History' are at the top. On the left, there are links for 'New Item', 'Build History', 'Project Relationship', and 'Check File Fingerprint'. Below these are sections for 'Build Queue' (showing 'No builds in the queue.') and 'Build Executor Status' (showing '0/1'). The main area is titled 'Build History of Jenkins' and contains a table with the following data:

S	Build	Time Since ↑	Status
✓	api-gateway-dev #9	19 min	stable
✓	user-service-dev #4	23 min	stable
✓	request-service-dev #4	23 min	stable
✓	export-service-dev #4	23 min	stable
✓	eureka-server-dev #4	23 min	stable
✓	client-service-dev #4	23 min	stable
✓	audit-service-dev #4	23 min	stable
✓	auth-service-dev #14	24 min	stable
✓	frontend-dev #7	25 min	stable
✓	api-gateway-dev #8	28 min	back to normal

FIGURE A.2 – Vue d'ensemble des jobs Jenkins validés pour les services de l'environnement dev

Sur la partie qualité, une limite réseau a été identifiée : le webhook SonarQube vers Jenkins n'était pas joignable dans la topologie retenue. Le contournement appliqué a consisté à interroger l'API SonarQube depuis Jenkins pour attendre activement la fin du calcul de la *quality gate*. Cette solution a permis de garder un contrôle qualité effectif sans bloquer l'intégration.

A.6 Déploiement Kubernetes et séparation des environnements

Le cluster retenu est un k3s mono-noeud, choisi pour rester cohérent avec les ressources du laboratoire. Les manifests ont été organisés pour séparer :

- Un socle commun `k8s/base` ;
- Les dépendances d'infrastructure `k8s/infra` ;
- Les services applicatifs `k8s/services` ;
- Le frontend et l'exposition via Ingress.

Le déploiement a été automatisé avec un script dédié, `scripts/deploy-k8s.sh`, capable d'appliquer soit toute la pile, soit un service précis. Cette approche a été décisive pour le laboratoire, car elle limite les redéploiements inutiles lorsque le cluster est sous contrainte CPU ou mémoire.

Trois namespaces ont été retenus pour représenter la progression des livraisons :

- `bmce-pfe` pour `dev` ;
- `bmce-recette` pour `recette` ;
- `bmce-prod` pour `prod`.

Les points d'accès utilisés pendant le projet sont :

- `dev.bmce.local`
- `recette.bmce.local`
- `prod.bmce.local`

Le déploiement continu a été validé sur l'environnement `dev`, puis étendu progressivement vers les autres environnements en fonction des ressources disponibles sur l'homelab.

A.7 Supervision

La supervision a été construite autour de Prometheus et Grafana avec deux objectifs :

- Suivre l'état système des VMs importantes ;
- Vérifier la disponibilité HTTP/HTTPS des interfaces du laboratoire.

Deux familles de sondes ont été utilisées :

- `node-exporter` pour les métriques système ;
- `blackbox-exporter` pour les vérifications d'accessibilité des services Web.

Cette couche ne remplace pas une observabilité applicative fine, mais elle fournit déjà un tableau de bord utile pour la démonstration et pour l'exploitation du laboratoire.

Cette capture montre le tableau de bord Grafana utilisé pour vérifier rapidement la disponibilité des services et l'état général du laboratoire.

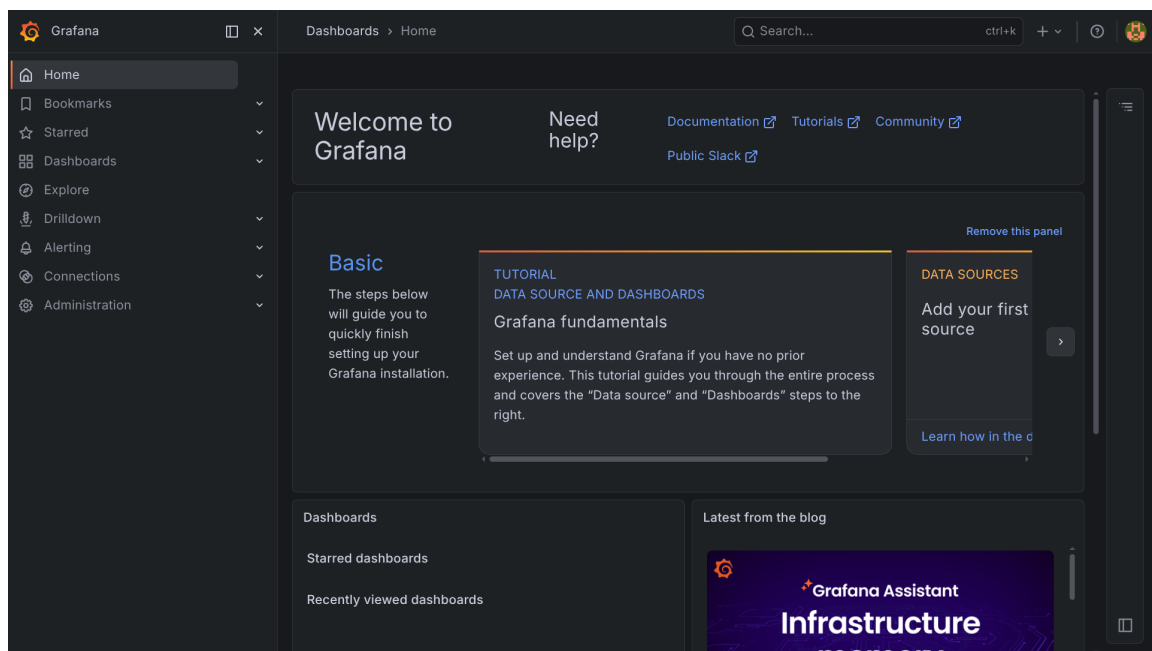


FIGURE A.3 – Tableau de bord Grafana utilisé pour suivre l'état général de la plateforme

A.8 Principaux incidents rencontrés

Plusieurs incidents ont eu une vraie valeur pédagogique pendant cette phase :

- Incident de démarrage du noyau Proxmox lié à la pile graphique ;
- Blocage du webhook SonarQube vers Jenkins ;
- Problèmes de probes et de configuration sur certains services Kubernetes ;
- Instabilité après redémarrage du cluster à cause de la résolution du registre Harbor ;
- Saturation mémoire progressive du cluster lors de l'activation simultanée de plusieurs environnements.

Ces incidents ont conduit à des corrections concrètes : durcissement du démarrage de l'hyperviseur, adaptation du pipeline Jenkins, stabilisation des manifests Kubernetes, ajout de vérifications post-déploiement et meilleure maîtrise de la consommation des environnements `dev`, `recette` et `prod`.

A.9 Bilan technique

Cette phase DevOps a permis d'aller au-delà d'une simple démonstration locale. Le projet dispose désormais d'un laboratoire cohérent, capable d'héberger la forge logicielle, la qualité, les artefacts, les images, les déploiements Kubernetes et la supervision.

Même si l'infrastructure reste un homelab de PFE et non une production réelle, elle démontre une chaîne CI/CD complète et crédible. Le résultat principal n'est donc pas seulement la présence de plusieurs outils, mais leur intégration effective dans un flux de livraison exploitable pour l'application BMCE.

Annexe B

Exemples Techniques

Cette annexe regroupe quelques éléments techniques utiles pour la compréhension et la reproduction de l'environnement de démonstration.

Pour le parcours mobile, les comptes clients de démonstration sont préparés de manière cohérente entre Keycloak, `auth-service` et `client-service`. Cette synchronisation garantit que le sujet Keycloak utilisé lors de l'authentification correspond bien au profil client exploité par l'application mobile.

Listing B.1 – Commandes utiles pour piloter la plateforme

```
1 ./start.sh
2 ./start.sh --quick
3 ./start.sh --status
4 ./start.sh --logs
5 ./start.sh --rebuild-restart request-service
6 ./start.sh --stop
```

Listing B.2 – Commandes de vérification principales

```
1 mvn -q -DskipTests compile
2 cd frontend && npm run build
```

Listing B.3 – Cycle de vie métier d'une demande

```
1 PENDING -> IN_PROGRESS -> APPROVED | REJECTED
```